

2017년도
학사학위논문

인터넷 공유기 펌웨어 업데이트 취약점 분석

Vulnerability analysis on Wireless Access Point Firmware Update

2017년 11월 13일

순천향대학교 공과대학
정보보호학과

이재현 고훈호
김창훈 송유정

인터넷 공유기 펌웨어 업데이트 취약점 분석

Vulnerability analysis on
Wireless Access Point Firmware Update

지도교수 유 일 선

이 논문을 공학사학위 논문으로 제출함

2017년 11월 13일

순천향대학교 공과대학
정보보호학과

이재현 고용호
김창훈 송유정

이 재 헌

고 용 호

김 창 훈

송 유 정 의 공학사학위논문을 인준함

2017년 11월 13일

<u>심 사 위 원</u>	<u>유일선 인</u>
----------------	--------------

<u>심 사 위 원</u>	<u>염홍열 인</u>
----------------	--------------

<u>심 사 위 원</u>	<u>임강빈 인</u>
----------------	--------------

<u>심 사 위 원</u>	<u>이선영 인</u>
----------------	--------------

<u>심 사 위 원</u>	<u>서정택 인</u>
----------------	--------------

순천향대학교 공과대학

정보보호학과

초 록

ICT(Information and Communication Technology)의 급속한 발전으로 인해 4차 산업혁명 시대가 도래하였으며, 이는 사물과 사물이 연결되어 통신하는 IoT(Internet of Things)의 등장 배경이 되었다. IoT는 스마트 오브젝트들이 인터넷에 상호 연결되어 데이터를 송·수신하는 환경으로 스마트 홈, 스마트 팩토리 그리고 스마트 오피스 등 많은 산업 환경에 적용되어 가고 있다. 사물과 사물이 인터넷에 연결되기 위해서는 인터넷 유·무선 공유기를 통한 연결이 필수적이며 이로 인해, 인터넷 유·무선 공유기는 IoT환경을 구성하기 위한 중요 요소이다. 하지만, 인터넷 유·무선 공유기의 경우 외부 접근이 비교적 쉽기 때문에 보안 취약점을 노린 악의적인 공격자에 의한 피해가 증가하고 있다. 인터넷 유·무선 공유기를 통해 많은 사물들이 인터넷에 연결되기 때문에 보안 취약점 문제를 해결하기 위한 노력이 시급하며, 본 논문에서는 이러한 시도의 일환으로 유·무선 공유기의 펌웨어를 수동으로 업데이트하는 과정에서 발생하는 취약점을 발견하고 이를 분석하였다. 유·무선 공유기의 펌웨어를 수동으로 업데이트하는 과정에서 변조된 펌웨어를 업데이트하여도 정상적으로 업데이트가 완료되는 취약점을 발견하였으며 이를 통해 발생할 수 있는 DoS(Denial of Service)공격을 재현하였다. 이러한 공격에 대응하기 위해서는 펌웨어 업데이트간의 암호화된 통신이 필요하며 본 논문에서 제시한 보안 프로토콜이 대응책으로 활용될 수 있다. 하지만 본 논문에서 제시한 보안 프로토콜들은 보안성 검증이 수행되어있지 않기 때문에 추후 연구로 이들을 정형화 검증하는 연구가 필요하다.

주요어 : IoT, 인터넷 공유기, Firmware Validation, 공유기 취약점

차 례

제 1 장 서론	1
제 2 장 연구배경	2
제 1 절 인터넷 공유기	2
제 2 절 펌웨어	4
제 3 절 스마트 업데이트	5
제 3 장 인터넷 공유기 펌웨어 업데이트 분석	9
제 1 절 인터넷 공유기 펌웨어 업데이트 개요	9
제 2 절 인터넷 공유기 환경	9
1. 인터넷 공유기 사양	10
2. 인터넷 공유기 펌웨어 구조	11
제 3 절 펌웨어 업데이트 분석	11
제 4 절 펌웨어 업데이트 공격 시나리오	41
제 4 장 안전한 펌웨어 업데이트 방법	46
제 5 장 결론	48
감사의 글	51

표 차 례

[표 2-1] 무선랜 보안기술	4
[표 2-2] 스마트 업데이트 메커니즘 비교	6
[표 3-1] X 제품 사양	10
[표 3-2] set_upgrade_mode(1) 함수 명령어	20
[표 3-3] TRX header 구조	37

그 립 차 례

[그림 2-1] 인터넷 공유기 판매 비율	2
[그림 2-2] 일반적인 무선랜 구성	4
[그림 2-3] 모바일 디바이스 정보	5
[그림 3-1] 2014년 컴퓨존 인터넷 공유기 시장 점유율	9
[그림 3-2] X 제품 펌웨어 구조	10
[그림 3-3] X 제품 펌웨어 추출	12
[그림 3-4] 펌웨어 업로드 패킷 분석	13
[그림 3-5] get_sw_upgrade_status() 함수 호출	13
[그림 3-6] get_sw_upgrade_status() 함수 호출된 파일 검색	14
[그림 3-7] get_sw_upgrade_status() 함수 일부	14
[그림 3-8] switch문 일부	15
[그림 3-9] set_sw_upgrade_status() 함수 호출	15
[그림 3-10] set_sw_upgrade_status() 함수 호출된 파일 검색	16
[그림 3-11] set_sw_upgrade_status() 함수 일부	16
[그림 3-12] write_body() 초반부	17
[그림 3-13] sub_411D48() 함수 호출	18
[그림 3-14] sub_411D48() 함수 일부	18
[그림 3-15] sub_411F38() 함수 호출	19
[그림 3-16] set_upgrade_mode(1) 함수 호출	20
[그림 3-17] unlink(/tmp/firmware) 함수 호출	21
[그림 3-18] set_sw_upgrade_status(1) 함수 호출	21
[그림 3-19] fopen(), fwrite(), fclose() 함수 호출	22
[그림 3-20] set_sw_upgrade_status(2), fputs(), kill(1, 2) 함수 호출	22
[그림 3-21] init 파일 위치	23
[그림 3-22] 명령어 비교문	23
[그림 3-23] sub_40F270() 함수 - signal_init(), signal() 함수 호출	24
[그림 3-24] sub_40E704() 함수(1)	25
[그림 3-25] sub_40E704() 함수(2)	26
[그림 3-26] sub_40E704() 함수(3)	26
[그림 3-27] sub_40E600(1) 함수	27
[그림 3-28] sub_40F270() 함수(2)	27
[그림 3-29] sub_430E674() 함수	28

[그림 3-30] sub_40F270() 함수(3)	29
[그림 3-31] unk_433970 switch table과 global point 주소	30
[그림 3-32] sub_40F270() 함수(4)	30
[그림 3-33] updaemon() 함수	31
[그림 3-34] sub_408380() 함수(1)	32
[그림 3-35] sub_408380() 함수(2)	32
[그림 3-36] updaemon() 함수(1)	33
[그림 3-37] hwinfo_get_bootloader_size() 함수가 호출된 파일 검색	33
[그림 3-38] hwinfo_get_bootloader_size() 함수 일부	34
[그림 3-39] updaemon() 함수(2)	34
[그림 3-40] sub_408570() 함수(1)	35
[그림 3-41] sub_408570() 함수(2)	36
[그림 3-42] safe_fread() 함수가 반환하는 값	36
[그림 3-43] sub_408570() 함수(3)	37
[그림 3-44] sub_408570() 함수(4)	38
[그림 3-45] package header 영역	38
[그림 3-46] sub_408570() 함수(5)	39
[그림 3-47] package header 비교 대상 필드	39
[그림 3-48] sub_408570() 함수(6)	40
[그림 3-49] sub_408570() 함수(7)	40
[그림 3-50] sub_408570() 함수(8)	41
[그림 3-51] sub_408570() 함수(9)	42
[그림 3-52] sub_408570() 함수(10)	42
[그림 3-53] shell을 통한 DoS 공격	43
[그림 3-54] 공유기 재부팅 관련 로그	43
[그림 3-55] 로그 AP 공격원리	45
[그림 4-1] 경북대 이임영 교수가 제안한 펌웨어 업데이트 프로토콜	46

제 1 장 서 론

ICT(Information and Communication Technology)의 급속한 발전으로 4차 산업혁명 시대가 다가오고 있다. 4차 산업혁명 시대에는 사물이 인터넷을 통해서 연결되어 통신하는 초연결사회가 이루어질 전망이다. 4차 산업혁명시대의 주요 핵심 기술은 연결사회를 구축하는 IoT(Internet of Things)이며, 이는 스마트시티, 스마트홈 등 다양한 산업 환경으로 적용되어 나가고 있다. 특히, IoT 환경의 다양한 기기들이 하나로 연결되기 위해서는 인터넷 유무선 공유기의 역할이 크다[1]. 즉, 스마트 플러그, 스마트 도어락과 같은 사물들이 네트워크에 연결되기 위해선 인터넷 유무선 공유기를 통한 인터넷 공급이 필수적이다.

하지만 사물인터넷의 수요가 증가하면서 그에 따른 피해가 급증하고 있다. 한국인터넷진흥원(KISA)의 조사에 따르면 지난해 인터넷침해대응센터에 접수된 IoT 보안 취약점 신고 건수가 전년 대비 2.8배 증가한 352건이었으며, 이는 전체 취약점 신고 건수의 52%에 해당한다고 밝혔다. 이 중 취약점이 가장 많이 발견된 IoT 기기는 인터넷 유무선 공유기였다[5].

인터넷 유무선 공유기의 경우 외부 접근이 비교적 쉽기 때문에 취약점을 노린 악의적인 침입자에 의해 치명적인 공격을 당할 가능성이 높다[3][4]. 예를 들어, 비밀번호 없이 접속을 하거나 디폴트 비밀번호를 사용하는 등의 보안 문제가 있으며, 또한 웹 취약점을 이용한 파밍 공격이나 펌웨어의 취약점을 악용한 서비스 거부 공격 등이 빈번히 발생하고 있다[5].

이에 따라 인터넷 유무선 공유기 보안이 필요하고 이것을 보완하기 위한 연구가 시급하다. 이를 위한 일환으로 본 논문에서는 인터넷 공유기 시장 점유율 78%로 1위인 A사 제품을 활용하여 공유기 펌웨어 업데이트 과정의 취약점을 분석하고, 발견된 취약점을 악용하여 실현될 수 있는 공격을 연구하고 기존 암호프로토콜을 기반으로 새로운 보안 프로토콜을 제시한다.

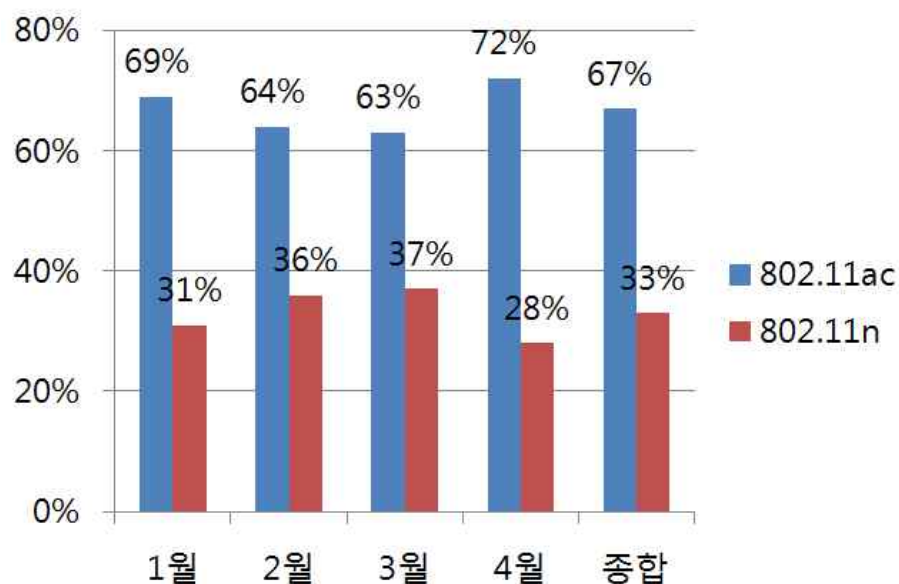
본 논문의 구성은 다음과 같다. 2장 연구 배경, 3장 인터넷 공유기 펌웨어 업데이트 분석, 4장 안전한 펌웨어 업데이트 방법, 5장에서 결론을 제시하며 본 논문을 마무리한다.

제 2 장 연구 배경

제 1 절 인터넷 공유기

인터넷 유무선 공유기란, 하나의 인터넷 회선을 이용해 여러 대의 기기들을 연결하는 매개체다. 이는 무선 네트워크 환경을 구축하기 위한 기기이며, WAP(Wireless Access Point)라고도 불린다. 인터넷 유무선 공유기의 연결 방식은 유선과 무선으로 나누어지며, 유선은 이더넷(Ethernet)으로, 무선은 Wi-Fi로 연결한다.

[그림 2-1]은 802.11ac와 802.11n 무선규약을 지원하는 공유기의 판매 비율을 나타낸다.



[그림 2-1] 인터넷 공유기 판매 비율[6]

최대 속도를 기준으로 802.11ac의 속도는 1300Mbps, 802.11n의 속도는 300Mbps를 보인다. 3년 전부터 802.11ac를 지원하는 유무선 공유기는 지속적으로 시장에 출시되어 많은 사람들이 폭넓게 제품을 선택할 수 있다. 시장에 나와있는 802.11ac를 지원하는 공유기는 802.11n 무선규약도 호환 가능하여

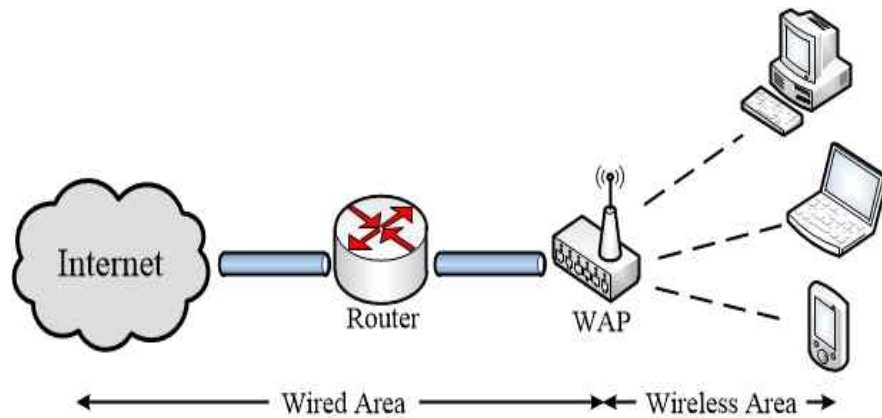
802.11n의 제품판매량은 줄어들고 있다. 반대로 802.11ac 제품의 판매가 늘어나는 추세이다. ICT기술 발전으로 인해 공유기 시장의 기술력은 더욱 발전될 것이며, 특히 4차 산업혁명의 핵심 중에 하나인 IoT 분야는 다양한 기기들과 연결되어 새로운 ICT 생태계를 구축할 것이다.

1.1 무선랜

무선랜이란 가정이나 공공장소에서 인터넷에 접속할 수 있도록 구축된 서비스를 의미한다. 무선랜은 무선 공유기(AP)가 설치되어 있는 장소에서 노트북이나 스마트폰 등의 단말을 이용하여 무선인터넷 서비스를 제공받을 수 있다. 서비스 반경은 최대 수백 미터로, 서비스 제공자 입장에서 무선랜은 초기 구축 시 많은 비용이 들지만 추후 배선을 관리·유지보수하는 비용이 유선랜보다 줄어들어 장기적인 관점에서는 효과적인 무선 네트워크 환경을 제공할 수 있다. 무선랜의 가장 큰 특징은 사용자의 이동성이 보장되면서 초고속 무선 네트워크 서비스를 제공할 수 있다는 것이다[4].

1.2 무선랜의 구성

무선랜은 물리적인 구성요소와 기술적인 구성요소로 구분할 수 있다. 먼저 무선 AP는 물리적인 구성요소로서, 유선랜의 마지막 부분에 위치하여 무선랜과 유선랜을 연결해주는 장치이다. 노트북, 스마트폰 등의 무선 단말기가 무선 AP로 사용된다. 무선랜 안테나 및 내장 또는 외장형 무선랜 카드를 포함하는 것은 무선 단말기라고 하며, 2개 이상의 무선랜을 연결하기 위해서는 무선 브릿지를 사용한다. 또한, 인증 서버 장비는 사용자의 인증을 한다. 기술적인 구성요소를 살펴보면, 32바이트로 이루어진 고유 식별자 SSID(Service Set Identifier)와 인증/암호화 요소가 있다. 무선 AP에서 지원하는 인증/암호화 기술은 WEP, WPA, WPA2 등으로 보안 강도에 따라 나뉘어지며, 안전하게 사용하기 위해서는 WPA2 설정을 해야한다.[5]. 일반적인 무선공유기를 이용한 무선랜 구성은 [그림1]과 같다[4].



[그림 2-2] 일반적인 무선랜 구성

1.3 무선랜 보안기술

무선 공유기에서 제공하는 인증/암호화 기술은 보안 강도에 따라 WEP, WPA, WPA2 등이 있다. 주요특징은 [표 1-1]과 같다[5].

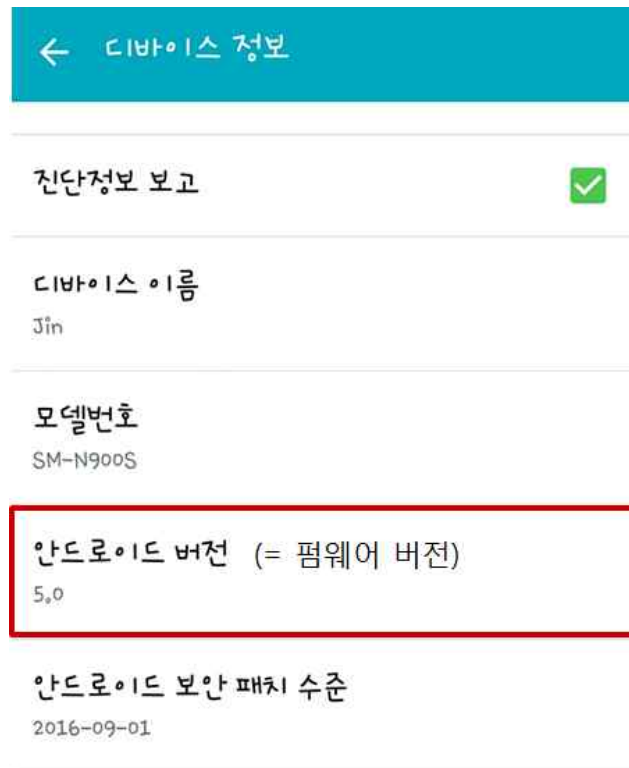
구분	WEP	WPA	WPA2
인증	공유된 (64비트, 128비트) 비밀키 사용	공유된 비밀키 사용 별도의 인증서버 이용	공유된 비밀키 사용 별도의 인증서버 이용
암호화	인증키와 동일한 고정 암호키 사용 RC4 알고리즘 사용	암호키 동적 변경(TKIP) RC4 알고리즘 사용	암호키 동적 변경 AES 등 강력한 블록 암호 알고리즘 사용
보안성	64비트 WEP 키 노출에 취약하여 자주 사용되지 않음	불완전한 RC4 알고리즘 사용 WEP 방식보다 안전	강력한 보안기능 제공

[표 2-1] 무선랜 보안기술

제 2절 펌웨어

펌웨어란, 롬(ROM)에 저장된 하드웨어를 제어하는 마이크로 프로그램을 의미하며, 일반적으로 하드웨어를 제어하는 소프트웨어를 펌웨어라고 한다. 펌웨어는 하드웨어와 소프트웨어의 두 특성 모두를 가지고 있으며, 프로그램 측면에서는 소프트웨어와 동일하지만 하드웨어와의 관계가 일반 응용소프트웨어와 구분된다. 하드웨어를 직접 제어하는 소프트웨어를 구현하면 분리된 소

프트웨어보다 속도가 증가하며, 비용적인 측면에서도 효율적으로 하드웨어를 제어 할 수 있다.



← 디바이스 정보	
진단정보 보고	✓
디바이스 이름	Jin
모델번호	SM-N900S
안드로이드 버전 (= 펌웨어 버전)	5.0
안드로이드 보안 패치 수준	2016-09-01

[그림 2-3] 모바일 디바이스 정보

[그림 2-3]에서 펌웨어 버전은 안드로이드 OS 버전을 뜻한다. 펌웨어 업그레이드를 통해 새로운 버전을 다운받아 기존 제품의 펌웨어를 업그레이드 한다. 제조사가 펌웨어를 배포함으로서 펌웨어를 업데이트할 수 있으며, 펌웨어 업데이트를 통해 품질향상 및 호환성 문제 개선이 가능하다. 즉, 펌웨어 업그레이드를 통해 각종 기기에 새로운 기능이나 콘텐츠를 추가하여 최신제품의 서비스 환경을 제공한다.

제 3절 스마트 업데이트

스마트 업데이트는 공급 업체로부터 제공되는 소프트웨어 혹은 펌웨어 같은 파일을 네트워크를 통해 다운로드 받아 업데이트하는 것을 말하며 이러한

업데이트는 업데이트 전에 발생하던 버그나 취약점 등을 쉽게 패치함으로써 인프라를 보호하는데 중요한 역할을 수행한다. 이러한 이유로 많은 업체들은 스마트 업데이트를 통해 소프트웨어 및 펌웨어 파일의 업데이트 파일을 배포하며 공급 업체마다 각기 다른 방법을 사용하여 업데이트 파일을 배포한다. 스마트 업데이트와 같은 업데이트 메커니즘은 이용자에게 안전하게 업데이트 파일을 배포하는 것이 가장 중요하다. 하지만 많이 사용되는 소프트웨어 업데이트 메커니즘은 신뢰할 수 있는 네트워크를 사용하는 것에 의존하고 있으며, 이는 중간자 공격 등의 발생을 유발할 수 있으므로 보안에 취약하다. [표 2-1]은 많이 사용되는 스마트 업데이트 메커니즘에 대한 비교분석 표이며 설명은 다음과 같다.

[표 2-2] 스마트 업데이트 메커니즘 비교

소프트웨어	플랫폼	인증된 연결 사용 여부
Apple Software Update	Mac OS	X
Windows Update	Windows	△
Adobe Acrobat	Mac OS	X
Microsoft Office	Mac OS	X
Mozilla Firefox	Windows	△
Fugu	Mac OS	X
McAfee VirusScan	Windows	X
McAfee VirusScan Enterprise	Windows	UnKnown
McAfee Virex	Mac OS	X
Debian	Linux	X

- Apple MacOS Software Update : 암호화된 연결이 필요 없고 Apple

Corp.에 의해 서명된 개인키에 대해서만 서명 가능하다. 각 업데이트에는 설치 실행파일 해시에 대한 1,024바이트의 서명이 포함된 "signature"이란 이름 포함되어있으며, Apple이 다운로드 서버를 관리하지 않는다.

- Microsoft Windows Update : Microsoft와 NTV를 지원하고 권장하는 카탈로그 파일을 통해 간접 서명된 동일한 Authenticode를 사용한다. 모든 카탈로그 파일은 Microsoft 루트와 연결된 RSA-SHA1 서명 체인이며, 텍스트 파일을 지원하지 않는 구성요소의 SHA1 해시가 포함되어 있고 RSA-SHA1으로 서명되어 있다. 클라이언트 측 WUcode는 업데이트를 설치하기 전에 모든 서명의 암호화 및 인증서 체인의 유효성을 검사한다.
- Adobe Acrobat Reader : 웹서버에서 업데이트된 업데이트 파일의 위치를 가져온 다음 업데이트 파일을 다운로드한다.
- Microsoft Office Update : Microsoft가 서명한 SW인증서는 다운로드 할 수 있는 각 업데이트에 대한 XML과 같은 파일에 포함된다. 업데이트를 위해 다운로드된 바이너리 파일은 해당 인증서 서명에 대해 확인하고 일치하지 않을 시 실행되지 않는다. Verisign의 Verification을 위한 Microsoft의 공개키는 설치 미디어에 포함되어 있다.
- Mozilla Firefox 1.0 & 1.5 : Firefox 1.0은 암호화되지 않은 HTTP를 통해 미러 서버에서 업데이트 패키지를 다운로드한다. Firefox 1.5는 의심스러운 인증서에 대해 경고한다. 그러나 업데이트 및 위치목록 자체는 인증되지 않아서 DNS Spoofing과 MITM 공격에 대처할 수 없다.
- Fugu : Spoofing된 DNS response나 XML 인터셉트 및 수정이 악의적인 코드를 유포할 수는 있지만 디스크 이미지만 사용자의 데스크톱에 탑재된다. 사용자가 악성 코드를 실행할 수 있지만 악성 응용 프로그램이 사용자로부터 루트 암호를 요청하여 얻지 않는 한, 사용자의 권한으로 제한된다.
- McAfeeVirusScan : 업데이트 프로세스를 시작하기 위해선 클라이언트는 구성요소 DLL, 실행파일, 바이러스 정의 파일의 버전 번호로 HTTP POST요청을 작성해야한다. 작성한 뒤, 서버가 생성한 응답을 분석해 구성요소가 유효한지 아닌지 판별하고 만일 유효하다면 클라이언트는 SW가

설치됐을 때 사용자가 승인한 Active X 구성요소의 동작을 제어하는 VBScript 지침을 포함하여 HTTP를 통해 서버에서 여러 문서를 요청한다.

- McAfee Virex 7 for Mac O : Virex 업데이트 관리자는 사용자에게 관리자 인증을 요청한 다음 McAfee의 FTP 서버에 익명으로 logon하고 특정 디렉토리에서 파일의 시간을 확인한다. 이 시간은 파일명에 부여되며 서버의 타임스탬프와는 무관하다. 업데이트 파일에는 임의의 코드를 포함할 수 있는 OS X설치 관리자 패키지가 포함되며 Virex는 이미 루트 액세스 권한을 얻었으므로 클라이언트는 검색 패키지를 설치한 후 백그라운드에서 설치 패키지를 실행한다.
- Debian Linux : Debian은 서명된 패키지를 배포한다.

제 3 장 인터넷 공유기 펌웨어 업데이트 분석

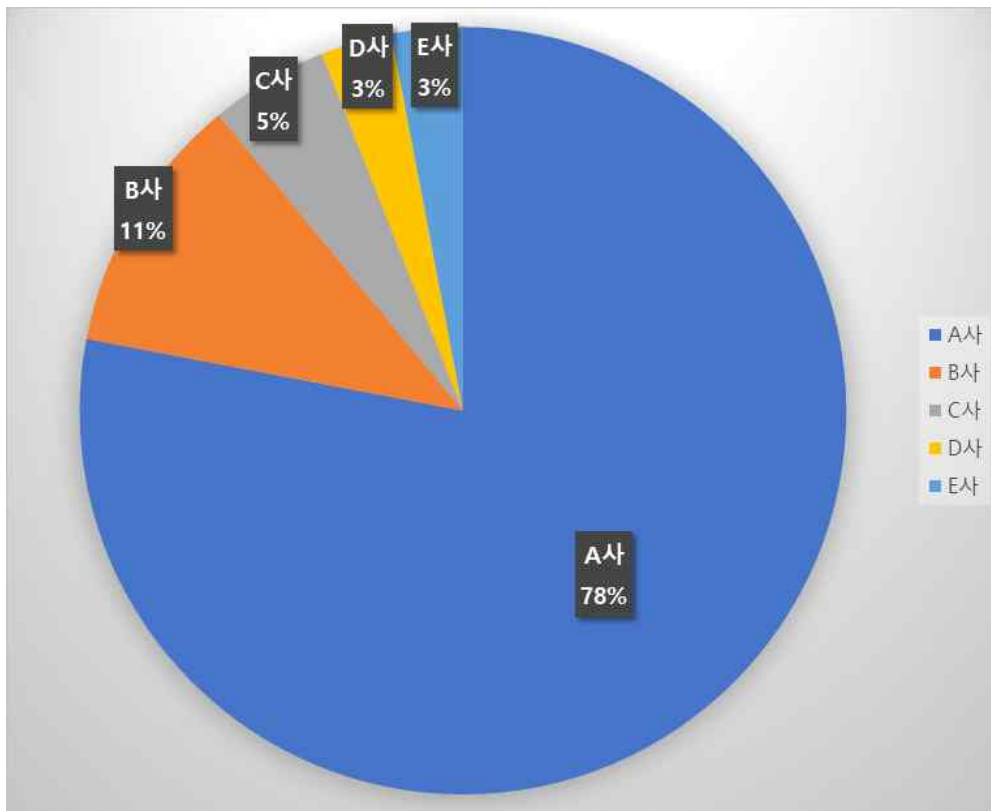
제 1절 인터넷 공유기 펌웨어 업데이트 개요

최근 인터넷 공유기의 보안이 대두되면서 몇몇 업체에서 펌웨어 업데이트 시 펌웨어의 유효성을 검사하는 절차가 강화되었다. 대표적으로 ASUS, TP-Link 업체이며 ASUS는 signature 업데이트를 도입해 전자서명을 유효성을 검사하는 기능을 도입했다. TP-Link는 RSA 서명을 이용해 펌웨어 유효성을 검사한다.

하지만 아직도 대다수의 인터넷 공유기 업체들은 위변조하기 쉬운 특정 값을 확인하는 과정만 펌웨어 유효성 검사에 도입했다. 본 논문에서는 실제 분석 사례를 통해 공유기 펌웨어 업데이트 취약점을 보이고 어떠한 공격으로 이어질 수 있는지를 보인다.

제 2절 인터넷 공유기 환경

본 논문에서 인터넷 공유기를 A사의 X 제품을 이용하여 무선 공유기 업데이트 취약점을 분석했다. A사를 선정한 된 배경은 2014년 국내 1위의 온라인 컴퓨터 쇼핑몰인 컴퓨존[7]에서 A사가 시장 점유율 78%로 압도적으로 1위를 차지하고 있다. [그림 3-1]은 상기의 시장 점유율을 보여준다.



[그림 3-1] 2014년 컴퓨존 인터넷 공유기 시장 점유율[8]

X 제품은 A사 보급형 제품 중 하나다. X 제품 펌웨어 버전은 2016년 11월 기준으로 최신 버전이며, 아래와 같은 단계로 분석했다.

- ① A사 인터넷 공유기 공식 홈페이지를 통해 펌웨어를 다운로드한다.
- ② firmware-mod-kit를 이용하여 펌웨어 분해한다.
- ③ wireshark[9]를 활용한 패킷 분석을 진행해서 펌웨어 업데이트 수행하는 파일을 추적한다.
- ④ IDA pro[10]을 통한 해부한 펌웨어 내 업데이트 기능을 수행하는 파일을 분석한다.

1. 인터넷 공유기 사양

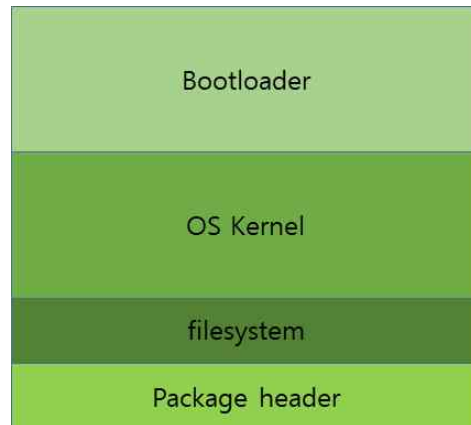
A사 X 제품의 제품사양은 [표 3-1]과 같다.

[표 3-1] X 제품 사양[11]

부품명	사양
SoC	Broadcom BCM5357
RAM	16MB
Flash Memory	4MB

2. 인터넷 공유기 펌웨어 구조

A사 X 제품의 펌웨어 구조는 [그림 3-2]과 같다.



[그림 3-2] X 제품 펌웨어 구조

1) 부트로더(Bootloader)

부트로더는 전기신호를 받은 뒤에 하드웨어 초기화를 진행하고, 디스크 내 Kernel image를 메모리에 옮긴다. 추출한 펌웨어는 CFE(Common Firmware Environment) bootloader를 사용한다. CFE는 Broadcom사의 32-bit, 64-bit의 System-On-Chip(SoC)시스템의 펌웨어 인터페이스와 bootloader 기능을 제공하기 위해 개발되었다[12].

2) Kernel

Kernel은 운영체제, 디바이스 드라이버가 존재하는 영역이다. B제품 펌웨어

에서는 TRX version1 포맷을 사용하며 TRX는 CFE bootloader를 사용하는 시스템에서 플래시 메모리에 Kernel image를 저장하기 위해 사용되는 포맷이다[13].

3) File System

File System은 파일 및 디렉토리를 효율적으로 관리할 수 있는 시스템이다. File System은 보통 압축형태로 구성되어 있으며, X 제품 펌웨어는 LZMA(Lempel-Ziv Markov chain algorithm) 압축을 사용하고, CramFS(Compressed ROM File System)로 구성되어 있다. LZMA는 데이터 압축에 쓰이는 알고리즘이며, 7-zip 압축 프로그램의 7z형식에 사용된다[14]. CramFS는 구조가 단순하며, 공간 효율성이 좋은 read-only 리눅스 파일 시스템이다[15].

4) Package header

Package header는 각 공유기 회사제품마다 다르다. 분석하고자 하는 A사의 펌웨어에서는 56bytes의 공간에 펌웨어 제품명, 버전 등이 기록되어 있다.

제 3절 펌웨어 업데이트 분석

인터넷 공유기 펌웨어 분석하기 위한 도구로는 Firmware-mod-kit, IDA pro, 그리고 Wireshark를 이용하였다. 펌웨어는 A사 공식 홈페이지를 통해 다운로드했으며, Firmware-mod-kit을 이용하여 파일 시스템을 추출하였다.

[그림 3-3]은 Firmware-mod-kit을 이용해 파일 시스템을 추출한 과정이다.

```

Scanning firmware...

Scan Time:      2017-08-15 16:45:13
Signatures:     193
Target File:    /home/span/Downloads/fmk/n604s_kr_9_972.bin
MD5 Checksum:   409c1111820a8b2ad0865d9b7aeb34c3

DECIMAL      HEX      DESCRIPTION
-----
6392         0x18F8   LZMA compressed data, properties: 0x03, dictionary size: 16777216 bytes, uncompressed size: 83951873 bytes
6432         0x1920   LZMA compressed data, properties: 0x0A, dictionary size: 16777216 bytes, uncompressed size: 201392385 bytes
6604         0x19CC   LZMA compressed data, properties: 0x0B, dictionary size: 16777216 bytes, uncompressed size: 234881792 bytes
45104        0xB030   LZMA compressed data, properties: 0x04, dictionary size: 16777216 bytes, uncompressed size: 570425344 bytes
46692        0xB664   LZMA compressed data, properties: 0x5D, dictionary size: 16777216 bytes, uncompressed size: 209044 bytes
131072       0x20000   TRX firmware header, little endian, header size: 28 bytes, image size: 3805184 bytes, CRC32: 0xAB048A1D flags/version: 0x10000
131100       0x2001C   LZMA compressed data, properties: 0x5D, dictionary size: 65536 bytes, uncompressed size: 4378624 bytes
1565632      0x17E3C0   CramFS filesystem, little endian size 2367488 version #2 sorted_dirs CRC 0x1c0c82a0, edition 0, 1973 blocks, 389 files

Extracting 1565632 bytes of header image at offset 0
Extracting cramfs file system at offset 1565632
Extracting 1024 byte footer from offset 3935288
Extracting CramFS file system...
Firmware extraction successful!

```

[그림 3-3] X 제품 펌웨어 추출

X 제품 펌웨어 파일은 공유기를 웹으로 제어하기 위한 cgi 파일, 공유기 설정 값이 존재하는 config 파일, 그리고 공유기 내부에서 동작하는 binary 파일로 구성되어 있다.

wireshark를 통해 펌웨어 업데이트에서 펌웨어가 공유기에 업로드하는 패킷을 추출했다. [그림 3-4]은 해당 패킷에 대한 상세 정보로 upgrade.cgi 파일이 펌웨어를 인터넷 공유기에 업로드하는 것을 보여준다.

```

> Frame 1370: 6377 bytes on wire (51016 bits), 6377 bytes captured (51016 bits) on interface 0
> Ethernet II, Src: KincoAut_7c:43:66 (00:23:55:7c:43:66), Dst: EfmNetwo_42:4b:56 (90:9f:33:42:4b:56)
> Internet Protocol Version 4, Src: 192.168.0.2, Dst: 192.168.0.1
> Transmission Control Protocol, Src Port: 2269, Dst Port: 80, Seq: 3931015, Ack: 1, Len: 6323
> [490 Reassembled TCP Segments (3937337 bytes): #382(694), #384(5840), #386(4380), #388(4380), #390(4380), #392(2920), #394(2920), #396
Hypertext Transfer Protocol
  > POST /sess-bin/upgrade.cgi HTTP/1.1\r\n
    Host: 192.168.0.1\r\n
    Connection: keep-alive\r\n
    Content-Length: 3936643\r\n
    Cache-Control: max-age=0\r\n
    Origin: http://192.168.0.1\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/55.0.2883.87 Safari/537.36\r\n
    Content-Type: multipart/form-data; boundary=----WebKitFormBoundary2dMuvD4tLxKYmfw\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n
    Referer: http://192.168.0.1/sess-bin/timepro.cgi?tmenu=sysconf&smenu=swupgrade\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: ko-KR,ko;q=0.8,en-US;q=0.6,en;q=0.4\r\n
    Cookie: efm_session_id=pl0W4Z7w4357cbNS\r\n
    \r\n
    [Full request URI: http://192.168.0.1/sess-bin/upgrade.cgi]
    [HTTP request 1/1]
    [Response in frame: 1441]
    File Data: 3936643 bytes
  MIME Multipart Media Encapsulation, Type: multipart/form-data, Boundary: "----WebKitFormBoundary2dMuvD4tLxKYmfw"
    [Type: multipart/form-data]
    First boundary: -----WebKitFormBoundary2dMuvD4tLxKYmfw\r\n
    Encapsulated multipart part: (application/octet-stream)
      Content-Disposition: form-data; name="upgrade"; filename="
      Content-Type: application/octet-stream\r\n\r\n
      > Data (3936312 bytes)
      Boundary: \r\n-----WebKitFormBoundary2dMuvD4tLxKYmfw\r\n
      Encapsulated multipart part:
      Last boundary: \r\n-----WebKitFormBoundary2dMuvD4tLxKYmfw\r\n

```

[그림 3-4] 펌웨어 업로드 패킷 분석

다음 [그림 3-5]은 upgrade.cgi 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 get_sw_upgrade_status() 함수를 호출한다. get_sw_upgrade_status() 함수가 호출되는 파일 검색 결과는 [그림 3-6]를 통해 나타내었다. 이어서, [그림 3-7]은 /lib/libcgi.so 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 get_sw_upgrade_status() 함수의 일부다. get_sw_upgrade_status() 함수는 /var/run/upgrade_status 파일을 통해 값을 읽어온다.

```

.text:00400F78      lw      $gp, 0x30+var_20($sp)
.text:00400F7C      nop
.text:00400F80      la      $t9, get_sw_upgrade_status
.text:00400F84      nop
.text:00400F88      jalr    $t9 ; get_sw_upgrade_status

```

[그림 3-5] get_sw_upgrade_status() 함수 호출

```
livefactor@livefactor-virtual-machine:~/study/firmware$ grep -rn get_sw_upgrade_status
Binary file rootfs/sbin/rc matches
Binary file rootfs/cgi-bin/upgrade.cgi matches
Binary file rootfs/lib/libcgi.so matches
```

[그림 3-6] get_sw_upgrade_status() 함수 호출된 파일 검색

```
.text:0001AA50      li      $gp, 0xB9CE0
.text:0001AA58      addu    $gp, $t9
.text:0001AA5C      addiu   $sp, -0x30
.text:0001AA60      sw      $gp, 0x30+var_20($sp)
.text:0001AA64      la      $a0, 0x70000
.text:0001AA68      nop
.text:0001AA6C      addiu   $a0, (aVarRunUpgrade_ - 0x70000) # "/var/run/upgrade_status"
.text:0001AA70      la      $a1, 0x70000
.text:0001AA74      nop
.text:0001AA78      addiu   $a1, (aR_6 - 0x70000) # "r"
.text:0001AA7C      sw      $s0, 0x30+var_10($sp)
.text:0001AA80      sw      $ra, 0x30+var_8($sp)
.text:0001AA84      sw      $gp, 0x30+var_C($sp)
.text:0001AA88      la      $t9, fopen
.text:0001AA8C      nop
.text:0001AA90      jalr    $t9 ; fopen
.text:0001AA94      nop
.text:0001AA98      nop
.text:0001AA9C      lw      $gp, 0x30+var_20($sp)
.text:0001AAA0      move    $s0, $v0
.text:0001AAA4      beqz    $s0, loc_1A9FC
.text:0001AAA8      move    $v0, $zero
.text:0001AAC0      nop
.text:0001AAC4      la      $a1, 0x70000
.text:0001AAC8      nop
.text:0001AACC      addiu   $a1, (aD_5 - 0x70000) # "%d"
.text:0001AAD0      addiu   $a2, $sp, 0x30+var_18
.text:0001AAD4      move    $a0, $s0
.text:0001AAD8      la      $t9, fscanf
.text:0001AAEC      nop
.text:0001AAF0      jalr    $t9 ; fscanf
```

[그림 3-7] get_sw_upgrade_status() 함수 일부

다음 [그림 3-8]은 upgrade.cgi 파일을 IDA pro를 이용한 정적 분석 과정에서 일부로, get_sw_upgrade_status() 함수에서 반환받은 값을 이용한 switch 문이다. [그림 3-9]은 upgrade.cgi 파일을 IDA pro를 이용한 정적 분석 과정에서 일부로, switch-case문의 일부다. 각 case문마다 set_sw_upgrade_switch() 함수를 호출한다.

.text:00401010	lw	\$gp, 0x30+var_20(\$sp)	
.text:00401020	sltiu	\$v0, \$s1, 0xB	
.text:00401024	sll	\$at, \$s1, 2	
.text:00401028	beqz	\$v0, loc_40161C	default문
.text:0040102C	move	\$v0, \$at	
.text:00401030	nop		
.text:00401034	la	\$at, 0x400000	
.text:00401038	nop		
.text:0040103C	addiu	\$at, (unk_401FC0 - 0x400000)	
.text:00401040	addu	\$at, \$v0	
.text:00401044	lw	\$v0, 0(\$at)	case문
.text:00401048	nop		
.text:0040104C	addu	\$v0, \$gp	
.text:00401050	jr	\$v0	
.text:00401054	nop		

[그림 3-8] switch문 일부

.text:004011C0	la	\$s0, 0x400000	
.text:004011C4	nop		
.text:004011C8	addiu	\$s0, (aSBt - 0x400000) # "%s %n"	
.text:004011CC	la	\$a1, 0x400000	
.text:004011D0	nop		
.text:004011D4	addiu	\$a1, (aRomIICAmAASmi - 0x400000) # 펌웨어 파일이 올바른지 검사합니다.	
.text:004011D8	move	\$a0, \$s0	
.text:004011DC	la	\$t9, printf	
.text:004011E0	nop		
.text:004011E4	jalr	\$t9 ; printf	
.text:004011E8	nop		
.text:004011EC	nop		
.text:004011F0	lw	\$gp, arg_10(\$sp)	
.text:004011F4	move	\$a0, \$s0	
.text:004011F8	la	\$a1, 0x400000	
.text:004011FC	nop		
.text:00401200	addiu	\$a1, (aUaLizmIldijmLn - 0x400000) # 잠시만 기다려주세요.	
.text:00401204	la	\$t9, printf	
.text:00401208	nop		
.text:0040120C	jalr	\$t9 ; printf	
.text:0040127C	la	\$t9, set_sw_upgrade_status	
.text:00401280	nop		
.text:00401284	jalr	\$t9 ; set_sw_upgrade_status	
.text:00401288	nop		
.text:0040128C	nop		

[그림 3-9] set_sw_upgrade_status() 함수 호출

[그림 3-10]은 set_sw_upgrade_status() 함수가 호출되는 파일 검색 결과를 나타내었다. 이어서, [그림 3-11]은 /lib/libcgi.so 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 set_sw_upgrade_status() 함수의 일부다. set_sw_upgrade_status() 함수는 /var/run/upgrade_status 파일 내 값을 수정하는 역할을 한다.

```
livefactor@livefactor-virtual-machine:~/study/firmware/rootfs$ grep -rn set_sw_upgrade_status
Binary file sbin/httpd matches
Binary file sbin/rc matches
Binary file cgibin/upgrade.cgi matches
Binary file lib/libuserland.so matches
```

[그림 3-10] set_sw_upgrade_status() 함수 호출된 파일 검색

```
.text:0000AF30      .globl set_sw_upgrade_status
.text:0000AF30      set_sw_upgrade_status:
.text:0000AF30      var_28          = -0x28
.text:0000AF30      var_20          = -0x20
.text:0000AF30      var_18          = -0x18
.text:0000AF30      var_14          = -0x14
.text:0000AF30      var_10          = -0x10
.text:0000AF30      var_C           = -0xC
.text:0000AF30      var_8           = -8
.text:0000AF30
.text:0000AF30      li             $gp, 0x4BD00
.text:0000AF38      addu           $gp, $t9
.text:0000AF3C      addiu          $sp, -0x38
.text:0000AF40      sw             $gp, 0x38+var_28($sp)
.text:0000AF44      sw             $s2, 0x38+var_10($sp)
.text:0000AF48      la             $a1, 0x10000
.text:0000AF4C      nop
.text:0000AF50      addiu          $a1, (off_E6F0 - 0x10000)
.text:0000AF54      move           $s2, $a0
.text:0000AF58      la             $a0, 0x10000
.text:0000AF5C      nop
.text:0000AF60      addiu          $a0, (aVarRunUpgrade_ - 0x10000) # "/var/run/upgrade_status"
.text:0000AF64      sw             $s1, 0x38+var_14($sp)
.text:0000AF68      sw             $ra, 0x38+var_8($sp)
.text:0000AF6C      sw             $gp, 0x38+var_C($sp)
.text:0000AF70      sw             $s0, 0x38+var_18($sp)
.text:0000AF74      la             $t9, fopen
.text:0000AF78      nop
.text:0000AF7C      jalr           $t9 ; fopen
.text:0000AF80      nop
.text:0000AF84      nop
.text:0000AF88      lw             $gp, 0x38+var_28($sp)
.text:0000AF8C      move           $s1, $v0
.text:0000AF90      li             $v0, 1
.text:0000AF94      beqz           $s1, loc_B044
```

[그림 3-11] set_sw_upgrade_status() 함수 일부

sbin/httpd, sbin/rc 파일이 set_sw_upgrade_status()를 사용했음을 [그림 3-10]를 통해 확인했다. [그림 3-12]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 write_body() 함수의 앞부분이다. write_body() 함수는 전역함수로 펌웨어를 공유기에 업로드하는 역할을 수행한다.

```

.text:004118CC      .globl write_body
.text:004118CC write_body:                                     # CODE XREF: process_requests+30C↓p
.text:004118CC                                     # DATA XREF: process_requests+304↓o ...
.text:004118CC      var_20      = -0x20
.text:004118CC      var_18      = -0x18
.text:004118CC      var_14      = -0x14
.text:004118CC      var_10      = -0x10
.text:004118CC      var_C       = -0xC
.text:004118CC      var_8       = -8
.text:004118CC
.text:004118CC      li        $gp, 0xFBF6B94
.text:004118D4      addu      $gp, $t9
.text:004118D8      addiu     $sp, -0x30
.text:004118DC      sw        $gp, 0x30+var_20($sp)
.text:004118E0      sw        $s2, 0x30+var_10($sp)
.text:004118E4      sw        $s1, 0x30+var_14($sp)
.text:004118E8      sw        $ra, 0x30+var_8($sp)
.text:004118EC      sw        $gp, 0x30+var_C($sp)
.text:004118F0      sw        $s0, 0x30+var_18($sp)
.text:004118F4      lw        $v0, 0x38($a0)
.text:004118F8      lw        $a3, 0x34($a0)
.text:004118FC      lw        $v1, 0x24($a0)
.text:00411900      subu     $s2, $v0, $a3
.text:00411904      lw        $a1, 0x20($a0)
.text:00411908      addu     $v0, $v1, $s2
.text:0041190C      sltu     $v0, $a1, $v0
.text:00411910      beqz     $v0, loc_411920
.text:00411914      move     $s1, $a0
.text:00411918      nop
.text:0041191C      subu     $s2, $a1, $v1

```

[그림 3-12] write_body() 초반부

다음, [그림 3-13]는 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 write_body() 함수 내에서 sub_411D48() 함수를 호출하는 과정이다. [그림 3-14]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 sub_411D48() 함수 일부다. sub_411D48() 함수는 특정 패킷에서 strstr() 함수를 통해 'form-data', 'name=' 이하 문자열을 검색해서 문자열이 존재 시 해당 문자열들을 반환한다. 이후, memcmp() 함수를 통해 'name=' 이후 문자열이 'upgrade'와 일치 여부를 확인한다.

```

.text:00411BEC loc_411BEC:                                # CODE XREF: write_body:loc_41199C↑j
.text:00411BEC      la      $t9, 0x410000
.text:00411BF0      nop
.text:00411BF4      addiu   $t9, (sub_411D48 - 0x410000)
.text:00411BF8      nop
.text:00411BFC      jalr    $t9 ; sub_411D48
.text:00411C00      nop
.text:00411C04      nop
.text:00411C08      lw      $gp, 0x30+var_20($sp)
.text:00411C0C      beqz    $v0, loc_4119A8
.text:00411C10      move    $a0, $s1
.text:00411C14      nop

```

[그림 3-13] sub_411D48() 함수 호출

```

.text:00411D84      nop
.text:00411D88      jalr    $t9 ; get_line
.text:00411D8C      nop
.text:00411D90      nop
.text:00411D94      lw      $gp, 0x30+var_20($sp)
.text:00411D98      nop
.text:00411D9C      la      $a1, 0x420000
.text:00411DA0      nop
.text:00411DA4      addiu   $a1, (aFormData - 0x420000) # "form-data"
.text:00411DA8      move    $a0, $s1
.text:00411DAC      la      $t9, strstr
.text:00411DB0      nop
.text:00411DB4      jalr    $t9 ; strstr
.text:00411DB8      nop
.text:00411DBC      nop
.text:00411DC0      lw      $gp, 0x30+var_20($sp)
.text:00411DC4      nop
.text:00411DC8      la      $a1, 0x420000
.text:00411DCC      nop
.text:00411DD0      addiu   $a1, (aName - 0x420000) # "name="
.text:00411DD4      beqz    $v0, loc_411EA4
.text:00411DD8      move    $a0, $v0
.text:00411DDC      nop
.text:00411DE0      la      $t9, strstr
.text:00411DE4      nop
.text:00411DE8      jalr    $t9 ; strstr
.text:00411DEC      nop
.text:00411DF0      nop
.text:00411DF4      lw      $gp, 0x30+var_20($sp)
.text:00411DF8      addiu   $a0, $v0, 6
.text:00411DFC      la      $a1, 0x420000
.text:00411E00      nop
.text:00411E04      addiu   $a1, (aUpgrade - 0x420000) # "upgrade"
.text:00411E08      li      $a2, 7
.text:00411E0C      beqz    $v0, loc_411EA4
.text:00411E10      nop
.text:00411E14      nop
.text:00411E18      la      $t9, memcmp
.text:00411E1C      nop
.text:00411E20      jalr    $t9 ; memcmp

```

[그림 3-14] sub_411D48() 함수 일부

sub_411D48() 함수가 끝나면 sub_411D48() 함수 호출이 종료된다. [그림 3-15]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 check_valid_session() 함수를 통해 세션이 유효한지 검사 후, sub_411F38() 함수를 호출한다. [그림 3-16]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 sub_411F38() 함수의 앞부분을 나타내었다. sub_411F38() 함수는 si_dram_size_mb() 함수를 통해 /default/var/run/si/pi 파일을 읽어서 config 값을 각각 메모리에 올리고, set_upgrade_mode(1) 함수를 호출한다. set_upgrade_mode() 함수는 /var/lib/libuserland.so 파일에 정의되어 있으며, 파라미터 값이 1인 경우 아래 [표 3-2]와 같은 명령어를 실행한다.

text:00411C10	move	\$a0, \$s1
text:00411C14	nop	
text:00411C18	la	\$t9, check_valid_session
text:00411C1C	nop	
text:00411C20	jalr	\$t9 ; check_valid_session
text:00411C24	nop	
text:00411C28	nop	
text:00411C2C	lw	\$gp, 0x30+var_20(\$sp)
text:00411C30	beqz	\$v0, loc_4119A8
text:00411C34	nop	
text:00411C38	nop	
text:00411C3C	la	\$t9, 0x410000
text:00411C40	nop	
text:00411C44	addiu	\$t9, (sub_411F38 - 0x410000)
text:00411C48	nop	
text:00411C4C	jalr	\$t9 ; sub_411F38

[그림 3-15] sub_411F38() 함수 호출

```

.text:00411F38 sub_411F38:                                     # CODE XREF: write_body+380↑p
.text:00411F38                                             # DATA XREF: write_body+378↑o
.text:00411F38 var_10 = -0x10
.text:00411F38 var_8 = -8
.text:00411F38 var_4 = -4
.text:00411F38
.text:00411F38 li $gp, 0x8BF6528
.text:00411F40 addu $gp, $t9
.text:00411F44 addiu $sp, -0x20
.text:00411F48 sw $gp, 0x20+var_10($sp)
.text:00411F4C la $v0, 0x10000000
.text:00411F50 nop
.text:00411F54 addiu $v0, (dword_100002C0 - 0x10000000)
.text:00411F58 lw $v0, (dword_100002C0 - 0x100002C0)($v0)
.text:00411F5C sw $ra, 0x20+var_4($sp)
.text:00411F60 la $a0, 0x420000
.text:00411F64 nop
.text:00411F68 addiu $a0, (aSecondRequest_ - 0x420000) # "second request.\n"
.text:00411F6C sw $gp, 0x20+var_8($sp)
.text:00411F70 bnez $v0, loc_411FF8
.text:00411F74 nop
.text:00411F78 nop
.text:00411F7C li $v0, 1
.text:00411F80 la $at, 0x10000000
.text:00411F84 nop
.text:00411F88 addiu $at, (dword_100002C0 - 0x10000000)
.text:00411F8C sw $v0, (dword_100002C0 - 0x100002C0)($at)
.text:00411F90 la $t9, si_dram_size_mb
.text:00411F94 nop
.text:00411F98 jalr $t9 ; si_dram_size_mb
.text:00411FD4 loc_411FD4: # CODE XREF: sub_411F38+78↑j
.text:00411FD4 la $t9, set_upgrade_mode
.text:00411FD8 nop
.text:00411FDC jalr $t9 ; set_upgrade_mode
.text:00411FE0 nop
.text:00411FE4 nop
.text:00411FE8 lw $gp, 0x20+var_10($sp)
.text:00411FEC b loc_411FBC
.text:00411FF0 nop
.text:00411FF4 # -----

```

[그림 3-16] set_upgrade_mode(1) 함수 호출

[표 3-2] set_upgrade_mode(1) 함수 명령어

echo 1 >> /var/run/upgrade_mode
killall apcpd
killall upnpd
killall timed
killall iptables-q
iptables -t nat -F tigger
killall igmpproxy
killall eapd
killall nas
killall wps_monitor
killall pptpd
killall dhclient
kill_process(dhclient.vlan2)
kill_process(iptableq)
kill_process(sysd)

set_upgrade_mode() 함수 호출을 마치면 sub_4111D48() 함수 호출이 종료된다. 다음, [그림 3-17]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 write_body() 함수에서 unlink(/tmp/firmware) 함수를 호출한다. 이를 통해 /tmp/firmware 파일을 삭제한다. [그림 3-18]은 “rm -rf /tmp/lost+found” 명령어를 실행하고 set_sw_upgrade_status(1) 함수를 호출한다.

.text:00411C60	la	\$a0, 0x420000
.text:00411C64	nop	
.text:00411C68	addiu	\$a0, (aTmpFirmware - 0x420000) # "/tmp/firmware"
.text:00411C6C	li	\$v0, 1
.text:00411C70	la	\$at, upgrade_start
.text:00411C74	nop	
.text:00411C78	sw	\$v0, (upgrade_start - 0x10000A78)(\$at)
.text:00411C7C	la	\$t9, unlink
.text:00411C80	nop	
.text:00411C84	jalr	\$t9 ; unlink

[그림 3-17] unlink(/tmp/firmware) 함수 호출

.text:00411CA0	addiu	\$a0, (aRmRfTmpLostFou - 0x420000) # "rm -rf /tmp/lost+found"
.text:00411CA4	la	\$t9, system
.text:00411CA8	nop	
.text:00411CAC	jalr	\$t9 ; system
.text:00411CB0	nop	
.text:00411CB4	nop	
.text:00411CB8	lw	\$gp, 0x30+var_20(\$sp)
.text:00411CBC	li	\$a0, 1
.text:00411CC0	la	\$t9, set_sw_upgrade_status
.text:00411CC4	nop	
.text:00411CC8	jalr	\$t9 ; set_sw_upgrade_status

[그림 3-18] set_sw_upgrade_status(1) 함수 호출

다음, [그림 3-19]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 fopen("/tmp/firmware", "a"), fwrite(), fclose() 함수를 호출한다. 이어서, [그림 3-20]은 sbin/httpd 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 set_sw_upgrade_status(2) 함수와 fputs(aUploadComplete, stderr) 함수, 그리고 kill(1, 2) 함수를 호출한다. 이 과정을 통해 콘솔에 펌웨어 업로드를 완료하는 내용을 출력하고 PID(Process Identifier)가 1인 프로세스에 SIGKILL 시그널을 전송한다.

```

.text:004119C4      la      $a0, 0x420000
.text:004119C8      nop
.text:004119CC      addiu   $a0, (aTmpFirmware - 0x420000) # "/tmp/firmware"
.text:004119D0      la      $a1, 0x420000
.text:004119D4      nop
.text:004119D8      addiu   $a1, (aA - 0x420000) # "a"
.text:004119DC      la      $t9, fopen
.text:004119E0      nop
.text:004119E4      jalr    $t9 ; fopen
.text:004119E8      nop
.text:004119EC      nop
.text:004119F0      lw      $gp, 0x30+var_20($sp)
.text:004119F4      lw      $a0, 0x34($s1)
.text:004119F8      move    $a2, $s2
.text:004119FC      move    $s0, $v0
.text:00411A00      li      $a1, 1
.text:00411A04      move    $a3, $v0
.text:00411A08      la      $t9, fwrite
.text:00411A0C      nop
.text:00411A10      jalr    $t9 ; fwrite
.text:00411A14      nop
.text:00411A18      nop
.text:00411A1C      lw      $gp, 0x30+var_20($sp)
.text:00411A20      move    $a0, $s0
.text:00411A24      move    $s2, $v0
.text:00411A28      la      $t9, fclose
.text:00411A2C      nop
.text:00411A30      jalr    $t9 ; fclose

```

[그림 3-19] fopen(), fwrite(), fclose() 함수 호출

```

.text:00411B28      li      $a0, 2
.text:00411B2C      beqz    $v0, loc_411A54

.text:00411B38      loc_411B38: # CODE XREF: write_body+22C1j
.text:00411B38      la      $t9, set_sw_upgrade_status
.text:00411B3C      nop
.text:00411B40      jalr    $t9 ; set_sw_upgrade_status
.text:00411B44      nop
.text:00411B48      nop
.text:00411B4C      lw      $gp, 0x30+var_20($sp)
.text:00411B50      nop
.text:00411B54      la      $a1, stderr
.text:00411B58      nop
.text:00411B5C      lw      $a1, (stderr - 0x10005D78)($a1)
.text:00411B60      la      $a0, 0x420000
.text:00411B64      nop
.text:00411B68      addiu   $a0, (aUploadComplete - 0x420000) # "upload complete.."
.text:00411B6C      la      $at, upgrade_start
.text:00411B70      nop
.text:00411B74      sw      $zero, (upgrade_start - 0x10000A78)($at)
.text:00411B78      la      $t9, fputs
.text:00411B7C      nop
.text:00411B80      jalr    $t9 ; fputs
.text:00411B84      nop
.text:00411B88      nop
.text:00411B8C      lw      $gp, 0x30+var_20($sp)
.text:00411B90      li      $a0, 1
.text:00411B94      li      $a1, 2
.text:00411B98      la      $t9, kill
.text:00411B9C      nop
.text:00411BA0      jalr    $t9 ; kill

```

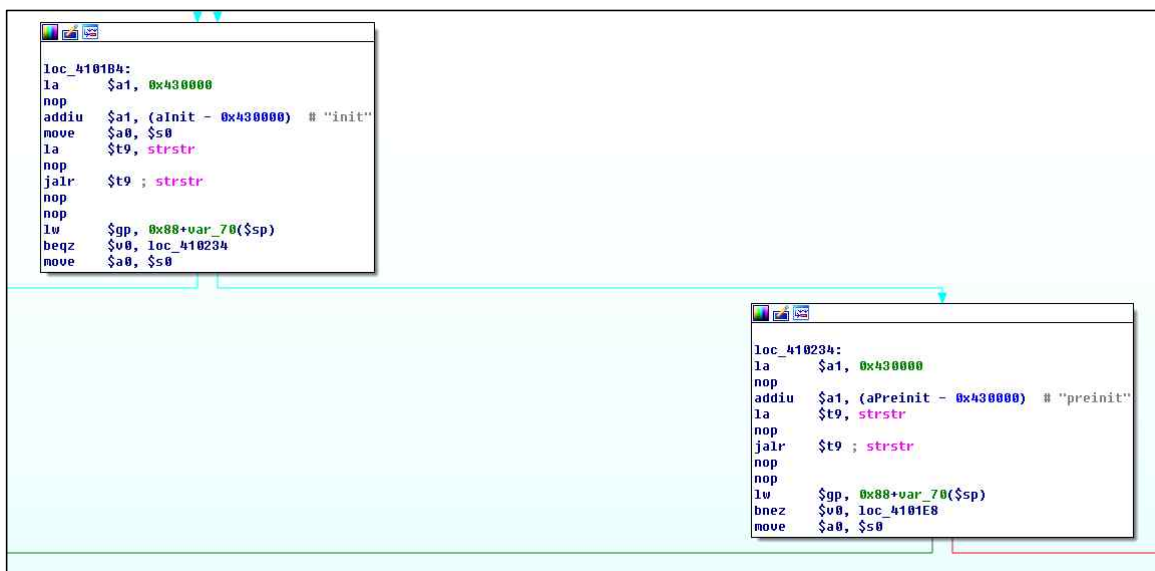
[그림 3-20] set_sw_upgrade_status(2), fputs(), kill(1, 2) 함수 호출

리눅스 OS에서 PID가 1인 프로세스는 init이다. init 프로세스는 리눅스가 부팅될 때 처음으로 생성되는 프로세스로 모든 프로세스의 부모 프로세스다. [그림 3-21]은 Firmware-mod-kit을 이용해서 파일시스템을 추출한 일부다. X 제품에서 init 파일은 /sbin 디렉토리에 존재하며 rc 파일에 심볼릭 링크된 파일임을 보여준다.

```
span@span-VirtualBox:~/Downloads/fmk/fmk/rootfs/sbin$ ls -al | grep init
lrwxrwxrwx  1 548 248      2 2017-08-15 16:45 init -> rc
```

[그림 3-21] init 파일 위치

rc 파일은 명령어 입력 시 문자열을 비교해서 init나 preinit 등의 시스템 프로세스가 실행되는 명령어를 감지하고, 각 프로세스에 맞게 명령을 수행하는 파일이다. [그림 3-22]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 init, preinit 명령어 비교문이다.



[그림 3-22] 명령어 비교문

rc 파일 내에서 시그널을 탐지하는 함수는 sub_40F270() 함수다. sub_40F270() 함수는 init, preinit 프로세스를 감지했을 때 실행하며 [그림

3-23]에서 보여준다. signal_init() 함수는 시그널 동작에 대한 초기화 과정을 진행하고 sub_40E704() 함수를 이용해 각 시그널 호출시 대응하는 함수들을 등록하는 과정이다. [그림 3-20]를 통해 kill(1,2) 함수를 실행했으므로 signal(2, sub_40E704()) 함수가 호출된다. [그림 3-24]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 sub_40E704() 함수 초반부다. sub_40E704() 함수는 각 signal 동작을 정의했으며 signal(2, sub_40E704()) 함수 호출시 loc_40EA38 위치로 이동한다.

```
.text:0040F3F4      addiu    $s0, (sub_40E704 - 0x410000)
.text:0040F43C      la      $t9, signal_init
.text:0040F440      nop
.text:0040F444      jalr    $t9 ; signal_init
.text:0040F448      nop
.text:0040F44C      nop
.text:0040F530      lw      $gp, 0xA8+var_98($sp)
.text:0040F534      li      $a0, 2
.text:0040F538      move    $a1, $s0
.text:0040F53C      la      $t9, signal
.text:0040F540      nop
.text:0040F544      jalr    $t9 ; signal
.text:0040F548      nop
.text:0040F54C      nop
.text:0040F550      lw      $gp, 0xA8+var_98($sp)
.text:0040F554      li      $a0, 0x19
.text:0040F558      move    $a1, $s0
.text:0040F55C      la      $t9, signal
.text:0040F560      nop
.text:0040F564      jalr    $t9 ; signal
.text:0040F568      nop
.text:0040F56C      nop
```

[그림 3-23] sub_40F270() 함수 - signal_init(), signal() 함수 호출

```

.text:0040E704 sub_40E704:                                     # DATA XREF: sub_40F270+184↓o
.text:0040E704
.text:0040E704 var_18 = -0x18
.text:0040E704 var_10 = -0x10
.text:0040E704 var_C = -0xC
.text:0040E704 var_8 = -8
.text:0040E704 var_4 = -4
.text:0040E704
.text:0040E704 li $gp, 0xFC7BC30
.text:0040E70C addu $gp, $t9
.text:0040E710 addiu $sp, -0x28
.text:0040E714 sw $gp, 0x28+var_18($sp)
.text:0040E718 sw $s0, 0x28+var_10($sp)
.text:0040E71C li $s0, 1
.text:0040E720 sw $ra, 0x28+var_4($sp)
.text:0040E724 sw $gp, 0x28+var_8($sp)
.text:0040E728 sw $s1, 0x28+var_C($sp)
.text:0040E72C beq $a0, $s0, loc_40EE54
.text:0040E730 nop
.text:0040E734 nop
.text:0040E738 li $v0, 0x19
.text:0040E73C li $at, 0x15
.text:0040E740 beq $a0, $v0, loc_40EDBC
.text:0040E744 move $v0, $at
.text:0040E748 nop
.text:0040E77C li $v0, 0x11
.text:0040E780 li $at, 2
.text:0040E784 beq $a0, $v0, loc_40EAE0
.text:0040E788 move $v0, $at
.text:0040E78C nop
.text:0040E790 li $at, 0xE
.text:0040E794 beq $a0, $v0, loc_40EA38
.text:0040E798 move $v0, $at
.text:0040E79C nop

```

\$v0 = 2

signal 2
분기점

[그림 3-24] sub_40E704() 함수(1)

다음, [그림 3-25]는 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 콘솔에 “Signalling Upgrade Start”를 출력하고 dword_10000088 변수에 1을 대입하는 과정을 보여준다. 이후, loc_40EA38로 이동해서 sub_40E600(1) 함수를 호출하며 [그림 3-26]에서 나타내었다.

```

.text:0040EA38 loc_40EA38:                                # CODE XREF: sub_40E704+90↑j
.text:0040EA38      la      $a0, 0x430000
.text:0040EA3C      nop
.text:0040EA40      addiu   $a0, (aDevConsole_1 - 0x430000) # "/dev/console"
.text:0040EA44      la      $a1, 0x430000
.text:0040EA48      nop
.text:0040EA4C      addiu   $a1, (aW_1 - 0x430000) # "w"
.text:0040EA50      la      $t9, fopen
.text:0040EA54      nop
.text:0040EA58      jalr    $t9 ; fopen
.text:0040EA5C      nop
.text:0040EA60      nop
.text:0040EA64      lw      $gp, 0x28+var_18($sp)
.text:0040EA68      beqz    $v0, loc_40EAC4
.text:0040EA6C      move    $s1, $v0
.text:0040EA70      nop
.text:0040EA74      la      $a0, 0x430000
.text:0040EA78      nop
.text:0040EA7C      addiu   $a0, (aSignallingUpgr - 0x430000) # "signalling Upgrade StartWn"
.text:0040EA80      li      $a1, 1
.text:0040EA84      li      $a2, 0x19
.text:0040EA88      move    $a3, $v0
.text:0040EA8C      la      $t9, fwrite
.text:0040EA90      nop
.text:0040EA94      jalr    $t9 ; fwrite
.text:0040EA98      nop
.text:0040EA9C      nop
.text:0040EAA0      lw      $gp, 0x28+var_18($sp)
.text:0040EAA4      move    $a0, $s1
.text:0040EAA8      la      $t9, fclose
.text:0040EAAC      nop
.text:0040EAB0      jalr    $t9 ; fclose
.text:0040EAB4      nop
.text:0040EAB8      nop
.text:0040EABC      lw      $gp, 0x28+var_18($sp)
.text:0040EAC0      nop
.text:0040EAC4 loc_40EAC4:                                # CODE XREF: sub_40E704+364↑j
.text:0040EAC4      la      $at, 0x10000000
.text:0040EAC8      nop
.text:0040EACC      addiu   $at, (dword_10000088 - 0x10000000)
.text:0040EAD0      sw      $s0, (dword_10000088 - 0x10000088)($at)
.text:0040EAD4      b       loc_40E7F4
.text:0040EAD8      nop

```

[그림 3-25] sub_40E704() 함수(2)

```

.text:0040E7F4 loc_40E7F4:                                # CODE XREF: sub_40E704+290↓j
.text:0040E7F4                                # sub_40E704+3D0↓j ...
.text:0040E7F4      | la      $a0, 0x10000000
.text:0040E7F8      | nop
.text:0040E7FC      | addiu   $a0, (dword_10000088 - 0x10000000)
.text:0040E800      | lw      $a0, (dword_10000088 - 0x10000088)($a0)
.text:0040E804      | la      $t9, 0x410000
.text:0040E808      | nop
.text:0040E80C      | addiu   $t9, (sub_40E600 - 0x410000)
.text:0040E810      | nop
.text:0040E814      | jalr    $t9 ; sub_40E600
.text:0040E818      | nop
.text:0040E81C      | nop
.text:0040E820      | lw      $gp, 0x28+var_18($sp)

```

[그림 3-26] sub_40E704() 함수(3)

sub_40E600() 함수는 파라미터 값을 queue에 저장하는 역할을 한다. [그림

3-27]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 sub_40E600() 함수다. 작업을 마치면 sub_40F270() 함수로 돌아와서 sub_40E694() 함수를 호출하는 과정을 [그림 3-28]에 나타내었다.

```
.text:0040E600 sub_40E600:                                     # CODE XREF: sub_40E704+110↓p
.text:0040E600                                     # DATA XREF: sub_40E704+108↓o
.text:0040E600      li      $gp, 0xFC7BD40
.text:0040E608      addu    $gp, $t9
.text:0040E60C      la      $v1, sigqueue_in
.text:0040E610      nop
.text:0040E614      lw      $v1, (sigqueue_in - 0x10082DB4)($v1)
.text:0040E618      nop
.text:0040E61C      addiu   $a1, $v1, 1
.text:0040E620      sll     $v1, 2
.text:0040E624      la      $at, signal_queue
.text:0040E628      nop
.text:0040E62C      addu    $at, $v1
.text:0040E630      sw      $a0, 0($at)
.text:0040E634      li      $v1, 0xA
.text:0040E638      la      $at, sigqueue_in
.text:0040E63C      nop
.text:0040E640      sw      $a1, (sigqueue_in - 0x10082DB4)($at)
.text:0040E644      beq     $a1, $v1, loc_40E65C
.text:0040E648      move    $v0, $zero
.text:0040E64C      nop
```

[그림 3-27] sub_40E600(1) 함수

```
.text:0040F828 loc_40F828:                                     # CODE XREF: sub_40F270+66C↓j
.text:0040F828                                     # sub_40F270+698↓j
.text:0040F828      la      $t9, 0x410000
.text:0040F82C      nop
.text:0040F830      addiu   $t9, (sub_40E674 - 0x410000)
.text:0040F834      nop
.text:0040F838      jalr    $t9 ; sub_40E674
.text:0040F83C      nop
.text:0040F840      nop
.text:0040F844      lw      $gp, 0xA8+var_98($sp)
.text:0040F848      nop
.text:0040F84C      la      $at, 0x10000000
.text:0040F850      nop
.text:0040F854      addiu   $at, (dword_10000084 - 0x10000000)
.text:0040F858      sw      $v0, (dword_10000084 - 0x10000084)($at)
.text:0040F85C      bne     $v0, $s0, loc_40F69C
```

[그림 3-28] sub_40F270() 함수(2)

sub_40E674() 함수에서 queue에 저장했던 값 1을 pop해서 v0 레지스터에 옮기는 역할을 수행하는 과정을 [그림 3-29]에 나타내었다. 이어서, [그림 3-30]와 같이 v0 레지스터 값을 .data 섹션에 존재하는 0x10000084 주소에 저

장한다. 이후, 0x10000084 주소값을 [그림 3-31]에서 unk_433970 switch table
를 참고하여 switch 분기문에 진입한다. switch table 값인 0xF03853AC과
global pointer 주소인 0x1008A340를 합하면 분기되는 주소를 찾을 수 있다.
계산 결과, 0x40F6EC 주소로 분기된다.

```

.text:0040E674 sub_40E674:                                     # CODE XREF: sub_40F270+5C8↓p
.text:0040E674                                     # DATA XREF: sub_40F270+5C8↓o
.text:0040E674      li      $gp, 0xFC7BCCC
.text:0040E67C      addu    $gp, $t9
.text:0040E680      la      $v1, sigqueue_out
.text:0040E684      nop
.text:0040E688      lw      $v1, (sigqueue_out - 0x10082DB8)($v1)
.text:0040E68C      la      $v0, sigqueue_in
.text:0040E690      nop
.text:0040E694      lw      $v0, (sigqueue_in - 0x10082DB4)($v0)
.text:0040E698      nop
.text:0040E69C      li      $a0, 0xFFFFFFFF
.text:0040E6A0      beq     $v0, $v1, locret_40E6E0
.text:0040E6A4      nop
.text:0040E6A8      nop
.text:0040E6AC      addiu    $v0, $v1, 1
.text:0040E6B0      sll     $v1, 2
.text:0040E6B4      la      $a0, signal_queue
.text:0040E6B8      nop
.text:0040E6BC      addu    $a0, $v1
.text:0040E6C0      lw      $a0, 0($a0)
.text:0040E6C4      li      $v1, 0xA
.text:0040E6C8      la      $at, sigqueue_out
.text:0040E6CC      nop
.text:0040E6D0      sw      $v0, (sigqueue_out - 0x10082DB8)($at)
.text:0040E6D4      beq     $v0, $v1, loc_40E6EC
.text:0040E6D8      nop
.text:0040E6DC      nop

.text:0040E6E0 locret_40E6E0:                               # CODE XREF: sub_40E674+2C↑j
.text:0040E6E0                                     # sub_40E674+84↓j
.text:0040E6E0      jr      $ra
.text:0040E6E4      move    $v0, $a0

```

\$a0 = 1

\$v0 = 1

[그림 3-29] sub_430E674() 함수

```

.text:0040F69C loc_40F69C:                                     # CODE XREF: sub_40F270+4C0↓j
.text:0040F69C                                             # sub_40F270+5EC↓j ...
.text:0040F69C      la      $v1, 0x10000000
.text:0040F6A0      nop
.text:0040F6A4      addiu   $v1, (dword_10000084 - 0x10000000)
.text:0040F6A8      lw      $v1, (dword_10000084 - 0x10000084)($v1)
.text:0040F6AC      nop
.text:0040F6B0      sltiu   $v0, $v1, 0xA
.text:0040F6B4      sll     $at, $v1, 2
.text:0040F6B8      beqz    $v0, loc_40F9EC
.text:0040F6BC      move    $v0, $at
.text:0040F6C0      nop
.text:0040F6C4      la      $v1, 0x430000
.text:0040F6C8      nop
.text:0040F6CC      addiu   $v1, (unk_433970 - 0x430000)
.text:0040F6D0      addu    $v1, $v0
.text:0040F6D4      lw      $v1, 0($v1)
.text:0040F6D8      nop
.text:0040F6DC      addu    $v1, $gp
.text:0040F6E0      jr      $v1

```

switch table

\$v3 = 0xF03853AC + 0x1008A340
= 0x40F6EC

그림 30 sub_40F270() 함수(3)

.rodata:00433970	unk_433970:	.byte 0xFC	#	# DATA XREF: sub_40F270+45C10
.rodata:00433971		.byte 0x53	# S	
.rodata:00433972		.byte 0x38	# 8	
.rodata:00433973		.byte 0xF0	#	
.rodata:00433974		.byte 0xAC	#	
.rodata:00433975		.byte 0x53	# S	
.rodata:00433976		.byte 0x38	# 8	
.rodata:00433977		.byte 0xF0	#	
.rodata:00433978		.byte 0xFC	#	
.rodata:00433979		.byte 0x55	# U	
.rodata:0043397A		.byte 0x38	# 8	
.rodata:0043397B		.byte 0xF0	#	
.rodata:0043397C		.byte 0x84	#	
.rodata:0043397D		.byte 0x54	# T	
.rodata:0043397E		.byte 0x38	# 8	
.rodata:0043397F		.byte 0xF0	#	
.rodata:00433980		.byte 0x30	# 0	
.rodata:00433981		.byte 0x56	# U	
.rodata:00433982		.byte 0x38	# 8	
.rodata:00433983		.byte 0xF0	#	
.rodata:00433984		.byte 0xD0	#	
.rodata:00433985		.byte 0x54	# T	
.rodata:00433986		.byte 0x38	# 8	
.rodata:00433987		.byte 0xF0	#	
.rodata:00433988		.byte 0x40	# @	
.rodata:00433989		.byte 0x54	# T	
.rodata:0043398A		.byte 0x38	# 8	
.rodata:0043398B		.byte 0xF0	#	
.rodata:0043398C		.byte 0x3C	# <	
.rodata:0043398D		.byte 0x57	# W	
.rodata:0043398E		.byte 0x38	# 8	
.rodata:0043398F		.byte 0xF0	#	
.rodata:00433990		.byte 0xC	#	
.rodata:00433991		.byte 0x58	# X	
.rodata:00433992		.byte 0x38	# 8	
abs:1008BD50	# Segment type: Absolute symbols			
abs:1008BD50	.globl _DYNAMIC			
abs:1008BD50	_DYNAMIC	= 0x400120		
abs:1008BD54	.globl _gp			
abs:1008BD54	_gp	= 0x1008A340		

[그림 3-31] unk_433970 switch table과 global point 주소

이후, updaemon("/tmp/firmware", "mtd1")를 실행하는 과정을 [그림 3-32]에 나타내었다. [그림 3-33]는 updaemon()은 가장 먼저 sub_408380()를 호출하는 과정이다.

.text:0040F6EC	la	\$a0, 0x430000
.text:0040F6F0	nop	
.text:0040F6F4	addiu	\$a0, (aTmpFirmware_0 - 0x430000) # "/tmp/firmware"
.text:0040F6F8	la	\$a1, 0x430000
.text:0040F6FC	nop	
.text:0040F700	addiu	\$a1, (aMtd1_0 - 0x430000) # "mtd1"
.text:0040F704	la	\$t9, updaemon
.text:0040F708	nop	
.text:0040F70C	jalr	\$t9 ; updaemon

[그림 3-32] sub_40F270() 함수(4)

```

.text:00409864      .globl updaemon
.text:00409864  updaemon:      # CODE XREF: sub_40F270+49C↓p
.text:00409864      # main+37C↓p
.text:00409864      # DATA XREF: ...
.text:00409864  var_68          = -0x68
.text:00409864  var_60          = -0x60
.text:00409864  var_58          = -0x58
.text:00409864  var_28          = -0x28
.text:00409864  var_24          = -0x24
.text:00409864  var_20          = -0x20
.text:00409864  var_1C          = -0x1C
.text:00409864  var_18          = -0x18
.text:00409864  var_14          = -0x14
.text:00409864  var_10          = -0x10
.text:00409864  var_C           = -0xC
.text:00409864  var_8           = -8
.text:00409864  var_4           = -4
.text:00409864
.text:00409864      li      $gp, 0xFC80ADC
.text:0040986C      addu    $gp, $t9
.text:00409870      addiu   $sp, -0x78
.text:00409874      sw      $gp, 0x78+var_68($sp)
.text:00409878      sw      $ra, 0x78+var_4($sp)
.text:0040987C      sw      $s7, 0x78+var_C($sp)
.text:00409880      sw      $s4, 0x78+var_18($sp)
.text:00409884      sw      $s3, 0x78+var_1C($sp)
.text:00409888      sw      $s2, 0x78+var_20($sp)
.text:0040988C      move    $s7, $a1
.text:00409890      move    $s4, $a0
.text:00409894      sw      $gp, 0x78+var_8($sp)
.text:00409898      sw      $s6, 0x78+var_10($sp)
.text:0040989C      sw      $s5, 0x78+var_14($sp)
.text:004098A0      sw      $s1, 0x78+var_24($sp)
.text:004098A4      sw      $s0, 0x78+var_28($sp)
.text:004098A8      la      $t9, 0x410000
.text:004098AC      nop
.text:004098B0      addiu   $t9, (sub_408380 - 0x410000)
.text:004098B4      nop
.text:004098B8      jalr    $t9 ; sub_408380

```

[그림 3-33] updaemon() 함수

다음, [그림 3-34]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 stat("/tmp/firmware") 함수를 통해 /tmp/firmware 경로에 업데이트할 펌웨어의 업로드 유무를 파악하는 것을 보여준다. 만약 /tmp/firmware 파일이 없을 경우 [그림 3-35]와 같이 콘솔에 "Firmware not found"를 출력한다.

```

.text:00408380 sub_408380:                                     # CODE XREF: updaemon+54↓p
.text:00408380                                     # .text:00409EC0↓p
.text:00408380                                     # DATA XREF: ...
.text:00408380 var_3B0 = -0x3B0 |
.text:00408380 var_3A8 = -0x3A8
.text:00408380 var_378 = -0x378
.text:00408380 var_318 = -0x318
.text:00408380 var_218 = -0x218
.text:00408380 var_18 = -0x18
.text:00408380 var_14 = -0x14
.text:00408380 var_10 = -0x10
.text:00408380 var_C = -0xC
.text:00408380 var_8 = -8
.text:00408380
.text:00408380 li $gp, 0xFC81FC0
.text:00408380 addu $gp, $t9
.text:00408380 addiu $sp, -0x3C0
.text:00408390 sw $gp, 0x3C0+var_3B0($sp)
.text:00408394 addiu $a1, $sp, 0x3C0+var_3A8
.text:00408398 sw $s0, 0x3C0+var_18($sp)
.text:0040839C sw $ra, 0x3C0+var_8($sp)
.text:004083A0 sw $gp, 0x3C0+var_C($sp)
.text:004083A4 sw $s2, 0x3C0+var_10($sp)
.text:004083A8 sw $s1, 0x3C0+var_14($sp)
.text:004083AC move $s0, $a0
.text:004083B0 la $t9, stat
.text:004083B4 nop
.text:004083B8 jalr $t9 ; stat
.text:004083BC nop
.text:004083C0 nop
.text:004083C4 lw $gp, 0x3C0+var_3B0($sp)
.text:004083C8 beqz $v0, loc_408430

```

[그림 3-34] sub_408380() 함수(1)

```

.text:004083D4 la $a3, stderr
.text:004083D8 nop
.text:004083DC lw $a3, (stderr - 0x1008C938)($a3)
.text:004083E0 la $a0, 0x430000
.text:004083E4 nop
.text:004083E8 addiu $a0, (aFirmwareNotFou - 0x430000) # "Firmware not found<br>"
.text:004083EC li $a1, 1
.text:004083F0 li $a2, 0x16
.text:004083F4 la $t9, fwrite
.text:004083F8 nop
.text:004083FC jalr $t9 ; fwrite
.text:00408400 nop
.text:00408404 nop
.text:00408408 lw $gp, 0x3C0+var_3B0($sp)
.text:0040840C li $v0, 0xFFFFFFFF

```

[그림 3-35] sub_408380() 함수(2)

이어서, [그림 3-36]와 같이 updaemon() 함수로 복귀해서 hwinfo_get_bootloader_size() 함수와 get_sw_upgrade_status() 함수를 호출한다. hwinfo_get_bootloader_size() 함수는 [그림 3-37]과 같이 /lib/libcgi.so에 정의되었다. [그림 3-38]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과

정 중 일부로 /var/run/hwinfo 파일을 통해 bootloader_size 값을 반환하는 과정이다.

```
.text:004098C4      lw      $gp, 0x78+var_68($sp)
.text:004098C8      move    $s2, $v0
.text:004098CC      la      $t9, hwinfo_get_bootloader_size
.text:004098D0      nop
.text:004098D4      jalr    $t9 ; hwinfo_get_bootloader_size
.text:004098D8      nop
.text:004098DC      nop
.text:004098E0      lw      $gp, 0x78+var_68($sp)
.text:004098E4      move    $s3, $v0
.text:004098E8      li      $v0, 0xFFFFFFFF
.text:004098EC      li      $s5, 0xFFFFFFFF
.text:004098F0      beq     $s2, $v0, loc_409D24
.text:004098F4      nop
.text:004098F8      nop
.text:004098FC      la      $t9, get_sw_upgrade_status
.text:00409900      nop
.text:00409904      jalr    $t9 ; get_sw_upgrade_status
.text:00409908      nop
.text:0040990C      nop
.text:00409910      lw      $gp, 0x78+var_68($sp)
.text:00409914      li      $s0, 3
.text:00409918      li      $s6, 0xFFFFFFFF
.text:0040991C      li      $s1, 5
.text:00409920      loc_409920: # CODE XREF: updaemon+114↓j
.text:00409920      la      $t9, get_sw_upgrade_status
.text:00409924      nop
.text:00409928      jalr    $t9 ; get_sw_upgrade_status
```

[그림 3-36] updaemon() 함수(1)

```
livefactor@livefactor-virtual-machine:~/study/firmware/rootfs$ grep -rn hwinfo_get_bootloader_size
Binary file sbin/rc matches
Binary file lib/libcgi.so matches
```

[그림 3-37] hwinfo_get_bootloader_size() 함수가 호출된 파일 검색

```

.text:000202A0 hwinfo_get_bootloader_size:
.text:000202A0
.text:000202A0 var_10      = -0x10
.text:000202A0 var_8       = -8
.text:000202A0 var_4       = -4
.text:000202A0
.text:000202A0          li      $gp, 0xB4490
.text:000202A8          addu   $gp, $t9
.text:000202AC          addiu  $sp, -0x20
.text:000202B0          sw     $gp, 0x20+var_10($sp)
.text:000202B4          la     $a0, 0x70000
.text:000202B8          nop
.text:000202BC          addiu  $a0, (aBootloader_siz - 0x70000) # "bootloader_size"
.text:000202C0          sw     $ra, 0x20+var_4($sp)
.text:000202C4          sw     $gp, 0x20+var_8($sp)
.text:000202C8          la     $t9, 0xD5B30
.text:000202CC          nop
.text:000202D0          jalr   $t9
.text:000202D4          nop
.text:000202D8          nop
.text:000202DC          lw     $gp, 0x20+var_10($sp)
.text:000202E0          move   $v1, $v0
.text:000202E4          li     $v0, 0xFFFFFFFF
.text:000202E8          li     $a0, 0xFFFFFFFF
.text:000202EC          beq    $v1, $v0, loc_202FC
.text:000202F0          nop
.text:000202F4          nop
.text:000202F8          move   $a0, $v1
.text:000202FC
.text:000202FC loc_202FC:          # CODE XREF: hwinfo_get_bootloader_size+4C↑j
.text:000202FC          lw     $ra, 0x20+var_4($sp)
.text:00020300          move   $v0, $a0
.text:00020304          addiu  $sp, 0x20
.text:00020308          jr     $ra

```

[그림 3-38] hwinfo_get_bootloader_size() 함수 일부

get_sw_upgrade_status() 함수를 통해 반환된 값이 3일 때 sub_408570() 함수를 호출한다. [그림 3-39]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 상기의 과정을 나타내었다.

```

.text:00409934          lw     $gp, 0x78+var_68($sp)
.text:00409938          move   $a0, $s4
.text:0040993C          addu   $a1, $s2, $s3
.text:00409940          addiu  $a2, $sp, 0x78+var_60
.text:00409944          beq    $v0, $s0, loc_409C30
.text:00409C30 loc_409C30:          # CODE XREF:
.text:00409C30          la     $t9, 0x410000
.text:00409C34          nop
.text:00409C38          addiu  $t9, (sub_408570 - 0x410000)
.text:00409C3C          nop
.text:00409C40          jalr   $t9 ; sub_408570 |

```

[그림 3-39] updaemon() 함수(2)

[그림 3-40]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부

로 sub_408570() 함수를 나타내었다. sub_408570() 함수는 펌웨어를 검증하는 함수로 먼저 fopen("/tmp/firmware", "r") 함수를 호출한다. 이후, [그림 3-41] 처럼 fseek(FILE *fp, 0x20000, 0) 함수와 safe_fread(var1, 1, 28, FILE *fp) 함수를 호출한다.

```
.text:00408570 sub_408570:                                     # CODE XREF: verify_firmware+60Jp
.text:00408570      li      $gp, 0xFC81DD0
.text:00408578      addu    $gp, $t9
.text:0040857C      addiu   $sp, -0x21E8
.text:00408580      sw      $gp, 0x21E8+var_21D0($sp)
.text:00408584      sw      $s2, 0x21E8+var_20($sp)
.text:00408588      move   $s2, $a1
.text:0040858C      la      $a1, 0x430000
.text:00408590      nop
.text:00408594      addiu   $a1, ($a1 - 0x430000) # "r"
.text:00408598      sw      $s3, 0x21E8+var_1C($sp)
.text:0040859C      sw      $s1, 0x21E8+var_24($sp)
.text:004085A0      sw      $s0, 0x21E8+var_28($sp)
.text:004085A4      sw      $ra, 0x21E8+var_8($sp)
.text:004085A8      sw      $gp, 0x21E8+var_C($sp)
.text:004085AC      sw      $s6, 0x21E8+var_10($sp)
.text:004085B0      sw      $s5, 0x21E8+var_14($sp)
.text:004085B4      sw      $s4, 0x21E8+var_18($sp)
.text:004085B8      move   $s3, $a2
.text:004085BC      move   $s0, $a0
.text:004085C0      la      $t9, fopen
.text:004085C4      nop
.text:004085C8      jalr    $t9 ; fopen
```

[그림 3-40] sub_408570() 함수(1)

```
.text:00407F7C      move   $a0, $s1
.text:00407F80      move   $a1, $s2
.text:00407F84      move   $a2, $zero
.text:00407F88      la      $t9, fseek
.text:00407F8C      nop
.text:00407F90      jalr    $t9 ; fseek
.text:00407F94      nop
.text:00407F98      nop
.text:00407F9C      lw      $gp, 0x21E8+var_21D0($sp)
.text:00407FA0      move   $a3, $s1
.text:00407FA4      addiu   $a0, $sp, 0x21E8+var_2188
.text:00407FA8      li      $a1, 1
.text:00407FAC      li      $a2, 0x1C
.text:00407FB0      la      $t9, safe_fread
.text:00407FB4      nop
.text:00407FB8      jalr    $t9 ; safe_fread
```

[그림 3-41] sub_408570() 함수(2)

safe_fread() 함수는 업로드한 펌웨어를 통해 [그림 3-41]의 값을 반환한다. [그림 3-42]의 값은 TRX header 영역이다. TRX header 구조는 [표 3]과 같다. safe_fread() 함수의 반환 값의 크기가 0x1C보다 크거나 같으면 magic number를 확인한다. magic number는 TRX header의 [3:0] 값이며 0x30524448 동일한지 확인한다. 상기의 과정을 [그림 3-43]를 통해 나타내었다.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00020000	48	44	52	30	00	10	3A	00	1D	8A	04	AB	00	00	01	00
00020010	1C	00	00	00	C0	E3	15	00	00	00	00	00	5D	00	00	01

[그림 3-42] safe_fread() 함수가 반환하는 값

[표 3-3] TRX header 구조[12]

0	1																2																3															
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1																	
magic number ('HDR0')																																																
length (header size + data)																																																
32-bit CRC value																																																
TRX flags																TRX version																																
Partition offset[0]																																																
Partition offset[1]																																																
Partition offset[2]																																																

.text:00408634	lw	\$gp, 0x21E8+var_21D0(\$sp)
.text:00408638	move	\$a3, \$v0
.text:0040863C	sltiu	\$v0, 0x1C
.text:00408640	lui	\$at, 0x3052
.text:00408644	bnez	\$v0, loc_408EA4
.text:00408648	move	\$v0, \$at
.text:0040864C	nop	
.text:00408650	lw	\$v1, 0x21E8+var_2188(\$sp)
.text:00408654	ori	\$v0, 0x4448
.text:00408658	lui	\$at, 0x400
.text:0040865C	beq	\$v1, \$v0, loc_408700
.text:00408660	move	\$v0, \$at

[그림 3-43] sub_408570() 함수(3)

[그림 3-44]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 fseek(FILE *fp, TRX_SIZE, 0) 함수와 fread(var2, 1, 0x38) 함수를 수행한다. fread() 함수의 반환 값은 [그림 3-45]와 같으며 함수 반환 값의 크기가 0x38과 동일한지 확인한다.

```
.text:00408718      sltiu    $v0, $v1, 0x1C
.text:0040871C      bnez     $v0, loc_40866C
.text:00408720      addu     $a1, $s2, $v1
.text:00408724      nop
.text:00408728      move     $a2, $zero
.text:0040872C      move     $a0, $s1
.text:00408730      la       $t9, fseek
.text:00408734      nop
.text:00408738      jalr     $t9 ; fseek
.text:0040873C      nop
.text:00408740      nop
.text:00408744      lw       $gp, 0x21E8+var_21D0($sp)
.text:00408748      move     $a0, $s3
.text:0040874C      li       $a1, 1
.text:00408750      li       $a2, 0x38
.text:00408754      move     $a3, $s1
.text:00408758      la       $t9, fread
.text:0040875C      nop
.text:00408760      jalr     $t9 ; fread
.text:00408764      nop
.text:00408768      nop
.text:0040876C      lw       $gp, 0x21E8+var_21D0($sp)
.text:00408770      move     $v1, $v0
.text:00408774      li       $v0, 0x38
.text:00408778      li       $a1, 1
.text:0040877C      beq      $v1, $v0, loc_4087C8
```

[그림 3-44] sub_408570() 함수(4)

	product_name	version	
003C1000	00 00 00	00 00 00	00 00 00
003C1010	52 54 4D 47 00 00 00 00	00 00 00 00 00 00 00 00	RTMG.....
003C1020	00 00 00 00 00 00 00 00	00 00 00 00 00 40 73 8A B5	@aŠu
003C1030	00 10 3A 00 4B E1 87 1D		...:Ká±.

[그림 3-45] package header 영역

다음, nvram_get_embedded() 함수를 호출하는 과정을 [그림 3-46]를 통해 나타내었다. nvram_get_embedded() 함수는 [그림 3-47]에 나타난 ‘product_name’, ‘no_downgrade’ 값을 반환하는 함수다.

```
.text:004087C8 loc_4087C8: # CODE XREF: sub_408570+20C7j
.text:004087C8      la      $a0, 0x430000
.text:004087CC      nop
.text:004087D0      addiu   $a0, (aProduct_name - 0x430000) # "product_name"
.text:004087D4      addiu   $a1, $sp, 0x21E8+var_21C8
.text:004087D8      la      $t9, nvram_get_embedded
.text:004087DC      nop
.text:004087E0      jalr    $t9 ; nvram_get_embedded
.text:004087E4      nop
.text:004087E8      nop
.text:004087EC      lw      $gp, 0x21E8+var_21D0($sp)
.text:004087F0      addiu   $s0, $sp, 0x21E8+var_168
.text:004087F4      bnez    $v0, loc_40882C

.text:0040882C loc_40882C: # CODE XREF: sub_408570+2847j
.text:0040882C      la      $a0, 0x430000
.text:00408830      nop
.text:00408834      addiu   $a0, (aNo_downgrade - 0x430000) # "no_downgrade"
.text:00408838      move    $a1, $s0
.text:0040883C      la      $t9, nvram_get_embedded
.text:00408840      nop
.text:00408844      jalr    $t9 ; nvram_get_embedded
.text:00408848      nop
.text:0040884C      nop
.text:00408850      lw      $gp, 0x21E8+var_21D0($sp)
.text:00408854      beqz    $v0, loc_4089C0
```

[그림 3-46] sub_408570() 함수(5)

00000A30	72 65 73 69 73 3D 35 00	70 72 6F 64 75 63 74 5F	resis=5.	product
00000A40	6E 61 6D 65 3D	00 74 66 74 70 5F	name=	.tftp_
00000A90	6F 3D 35 00	6E 6F 5F 64 6F 77 6E 67 72 61 64 65	o=5.	no downgrade
00000AA0	3D	00 00 00 00 00 00 00 00 00 00 00 00	=	

[그림 3-47] package header 비교 대상 필드

이후, [그림 3-48]은 /sbin/rc 파일을 IDA pro를 이용한 정적 분석 과정 중 일부로 펌웨어 버전을 확인하는 과정이다. [그림 3-45]의 ‘version’ 값과 [그림 3-47]의 ‘no_downgrade’ 문자열 이후 값을 확인한다. 이때 ‘version’ 값이 ‘no_downgrade’ 값보다 낮거나 같다면 펌웨어 업데이트를 진행하지 않는다. 버전 확인이 끝나면 제품명을 비교하는 과정을 [그림 3-49]를 통해 나타내었다.

```

.text:00408860      sb      $zero, 0x21E8+var_167($sp)
.text:00408864      la      $t9, atoi
.text:00408868      nop
.text:0040886C      jalr     $t9 ; atoi
.text:00408870      nop
.text:00408874      nop
.text:00408878      lw      $gp, 0x21E8+var_21D0($sp)
.text:0040887C      addiu   $s0, $sp, 0x21E8+var_148
.text:00408880      addiu   $a0, $sp, 0x21E8+var_166
.text:00408884      move    $s5, $v0
.text:00408888      la      $t9, atoi
.text:0040888C      nop
.text:00408890      jalr     $t9 ; atoi
.text:00408894      nop
.text:00408898      nop
.text:0040889C      lw      $gp, 0x21E8+var_21D0($sp)
.text:004088A0      addiu   $a1, $s3, 8
.text:004088A4      move    $a0, $s0
.text:004088A8      move    $s6, $v0
.text:004088AC      la      $t9, strcpy
.text:004088B0      nop
.text:004088B4      jalr     $t9 ; strcpy
.text:004088B8      nop
.text:004088BC      nop
.text:004088C0      lw      $gp, 0x21E8+var_21D0($sp)
.text:004088C4      move    $a0, $s0
.text:004088C8      sb      $zero, 0x21E8+var_147($sp)
.text:004088CC      la      $t9, atoi
.text:004088D0      nop
.text:004088D4      jalr     $t9 ; atoi
.text:004088D8      nop
.text:004088DC      nop
.text:004088E0      lw      $gp, 0x21E8+var_21D0($sp)
.text:004088E4      addiu   $a0, $sp, 0x21E8+var_146
.text:004088E8      move    $s0, $v0
.text:004088EC      la      $t9, atoi
.text:004088F0      nop
.text:004088F4      jalr     $t9 ; atoi
.text:004088F8      nop
.text:004088FC      nop
.text:00408900      lw      $gp, 0x21E8+var_21D0($sp)
.text:00408904      nop
.text:00408908      la      $a0, stderr
.text:0040890C      nop
.text:00408910      lw      $a0, (stderr - 0x1008C930)($a0)
.text:00408914      la      $a1, 0x430000
.text:00408918      nop
.text:0040891C      addiu   $a1, (aDowngradeIsNot - 0x430000) # "Downgrade is not allowed to %d.%d. And ..."
.text:00408920      move    $a2, $s5
.text:00408924      move    $a3, $s6
.text:00408928      move    $s4, $v0
.text:0040892C      sw      $s0, 0x21E8+var_21D0($sp)
.text:00408930      sw      $v0, 0x21E8+var_21D4($sp)
.text:00408934      la      $t9, fprintf
.text:00408938      nop
.text:0040893C      jalr     $t9 ; fprintf
.text:00408940      nop
.text:00408944      nop
.text:00408948      lw      $gp, 0x21E8+var_21D0($sp)
.text:0040894C      slt     $v0, $s0, $s5
.text:00408950      beqz    $s0, loc_408968
.text:00408954      nop
.text:00408958      nop
.text:0040895C      bnez    $v0, loc_4086A8

```

[그림 3-48] sub_408570() 함수(6)

```

.text:00408C70      loc_408C70:                                     # CODE XREF: sub_408570+7BC↓j
.text:00408C70      addiu   $a0, $sp, 0x21E8+var_21C8
.text:00408C74      move    $a1, $s3
.text:00408C78      la      $t9, strcmp
.text:00408C7C      nop
.text:00408C80      jalr     $t9 ; strcmp
.text:00408C84      nop
.text:00408C88      nop
.text:00408C8C      lw      $gp, 0x21E8+var_21D0($sp)
.text:00408C90      addiu   $a0, $sp, 0x21E8+var_217C
.text:00408C94      beqz    $v0, loc_408A14

```

[그림 3-49] sub_408570() 함수(7)

이어서, [그림 3-50]는 CRC(Cyclic Redundancy Check) Checksum 비교하기 위한 CRC32 Checksum 계산 과정이다. TRX header영역을 fseek() 함수를 통해 가져와 crc32(buf, 0x10, 0xffffffff) 함수로 CRC32 Checksum을 계산한다. [그림 3-51]은 0x2000크기만큼 fread()를 통해 가져와 crc32() 함수를 수행한다. 이 과정을 TRX의 마지막 영역까지 반복한다. 이렇게 계산한 Checksum과 [그림 3-42]의 CRC Checksum 영역이 일치한지 비교하는 과정을 [그림 3-52]을 통해 나타내었다. 만약 stat("/tmp/firmware") 과정부터 CRC32 비교까지 전 과정이 올바르게 진행되었다면 디바이스는 펌웨어가 정상임을 확인하고 펌웨어를 플래시 메모리에 write 작업을 수행한다.

```
.text:00408A14 loc_408A14:                                # CODE XREF: sub_408570+46C↑j
.text:00408A14                                # sub_408570+724↓j ...
.text:00408A14      li      $a1, 0x10
.text:00408A18      li      $a2, 0xFFFFFFFF
.text:00408A1C      la      $t9, crc32
.text:00408A20      nop
.text:00408A24      jalr    $t9 ; crc32
.text:00408A28      nop
.text:00408A2C      nop
.text:00408A30      lw      $gp, 0x21E8+var_21D0($sp)
.text:00408A34      addiu   $a1, $s2, 0x1C
.text:00408A38      move    $a0, $s1
.text:00408A3C      move    $a2, $zero
.text:00408A40      move    $s3, $v0
.text:00408A44      la      $t9, fseek
.text:00408A48      nop
.text:00408A4C      jalr    $t9 ; fseek
.text:00408A50      nop
.text:00408A54      nop
.text:00408A58      lw      $gp, 0x21E8+var_21D0($sp)
.text:00408A5C      lw      $v0, 0x21E8+var_2184($sp)
.text:00408A60      nop
.text:00408A64      addiu   $s0, $v0, -0x1C
.text:00408A68      blez    $s0, loc_408B00
```

[그림 3-50] sub_408570() 함수(8)

```
.text:00408B00 loc_408B00:                                # CODE XREF: sub_408570+4F8↑j
.text:00408B00                                # sub_408570+63C↓j ...
.text:00408B00      lw      $v0, 0x21E8+var_2180($sp)
.text:00408B04      nop
.text:00408B08      li      $a2, 0x38
.text:00408B0C      beq     $v0, $s3, loc_408B58
```

[그림 3-51] sub_408570() 함수(10)

```

text:00408A7C loc_408A7C: # CODE XREF: sub_408570+584↓j
text:00408A7C      li      $a1, 1
text:00408A80      li      $a2, 0x2000
text:00408A84      move    $a3, $s1
text:00408A88      la      $t9, fread
text:00408A8C      nop
text:00408A90      jalr     $t9 ; fread
text:00408A94      nop
text:00408A98      nop
text:00408A9C      lw      $gp, 0x21E8+var_21D0($sp)
text:00408AA0      move    $v1, $v0
text:00408AA4      addu    $a3, $s2, $v0
text:00408AA8      move    $a1, $v0
text:00408AAC      move    $a2, $s3
text:00408AB0      move    $a0, $s4
text:00408AB4      slt     $v0, $s0, $a3
text:00408AB8      blez    $v1, loc_408BB8
text:00408ABC      nop
text:00408AC0      nop
text:00408AC4      bnez    $v0, loc_408B90
text:00408AC8      nop
text:00408ACC      nop
text:00408AD0      move    $s2, $a3
text:00408AD4      la      $t9, crc32
text:00408AD8      nop
text:00408ADC      jalr     $t9 ; crc32
text:00408AE0      nop
text:00408AE4      nop
text:00408AE8      lw      $gp, 0x21E8+var_21D0($sp)
text:00408AEC      move    $s3, $v0
text:00408AF0      slt     $v0, $s2, $s0
text:00408AF4      bnez    $v0, loc_408A7C

```

[그림 3-52] sub_408570() 함수(9)

제 4절 펌웨어 업데이트 공격 시나리오

제 3절에서 분석한 펌웨어 업데이트 절차는 취약점이 존재한다. 펌웨어 파일이 암호화되지 않았으며 펌웨어 파일의 TRX Header, Package Header 영역 값을 변조한다면 얼마든지 펌웨어를 위변조할 수 있다. 이는 변조된 펌웨어를 통해 여러 가지 공격을 유발할 수 있다.

1) DoS 공격(Denial of Service)

공격자 관점에서 무선 공유기 펌웨어 업데이트 시나리오는 아래와 같다.

- ① 공유기 관리페이지에 로그인한다.
- ② 공유기 shell에 접근할 수 있는 변조된 펌웨어를 수동 업데이트한다.
- ③ 공격자는 공유기 shell에 접속하고, /tmp 디렉토리에 wget 명령어를 사

용해 DoS공격을 한다.

아래 [그림 3-53]은 공유기에 직접 DoS공격을 하는 과정이며 [그림 3-54]은 공유기가 다운되어 자동으로 재부팅되는 현상이다.

```
# cd /tmp
# /usr/bin/wget -O test_dos1 100% |*****| 1620 KB 00:00 ETA
# /usr/bin/wget -O test_dos2 100% |*****| 1620 KB 00:00 ETA
# /usr/bin/wget -O test_dos3 100% |*****| 1620 KB 00:00 ETA
# /usr/bin/wget -O test_dos4 100% |*****| 1620 KB 00:00 ETA
#
# /usr/bin/wget -O test_dos5 0% | | 0 --- ETA
```

[그림 3-53] shell을 통한 DoS 공격

```
telnetd: starting
Reaped 407
Reaped 302
port: 18; interface: any; login program: /bin/sh
Reaped 326
Run Reboot after 2 seconds
```

[그림 3-54] 공유기 재부팅 관련 로그

이를 통해 공유기는 네트워크 서비스를 원활히 실시할 수 없으며, 사용자는 큰 불편을 겪게 될 것이다.

2) 로그 AP(Rogue Access Point)

로그 AP는 악의적으로 목적으로 설치된 공유기를 통칭한다. 유형으로는 인가되지 않은 AP(Unmanaged Access Point), 이블 트윈(Evil Twin), 소프트 로그 AP(Soft Rogue Access Point)가 있다.

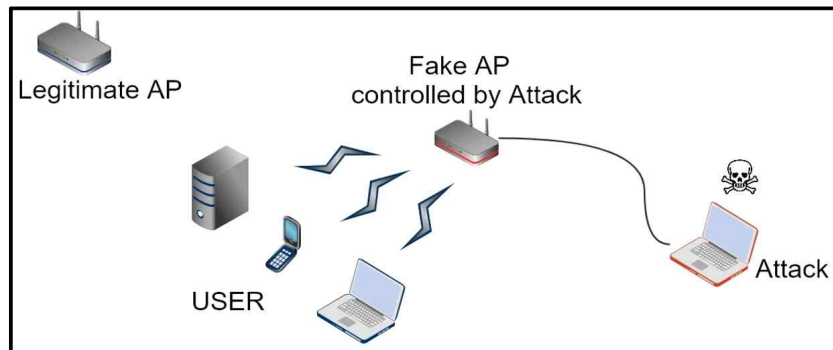
인가되지 않은 AP는 네트워크 내에 허가를 받지 않은 AP를 설치하는 것을

말한다. 공격자는 이를 통해 AP에 접속함으로써 네트워크 내부에 침투할 수 있다. 이 공격은 공격자가 인가되지 않은 AP를 설치하기 전에는 달리 대처할 방법이 없고, 네트워크 전체를 감시하는 보안장비나 소프트웨어가 없다면 설치된 후에도 감지 및 차단이 어렵다.

소프트 AP는 소프트웨어를 이용하여 가상으로 AP를 만들어 내는 것을 말한다. 소프트 AP는 여러 운영체제에서, 소프트웨어 도구들을 이용하여 만들어지며 브릿지(Bridge)를 통해 네트워크와 연결시키면 무선 AP와 같은 기능을 수행한다. 이 기법을 악용하여 이블 트윈과 같은 공격들을 시도할 때 이를 소프트 로그 AP라고 한다.

이블 트윈은 공격자가 정상 AP 주변에서 동일한 ESSID를 가지는 실제 혹은 소프트 AP를 설치한 것을 뜻한다. 공유기의 이름이 동일하면 사용자는 출력신호가 강한 쪽에 연결 되므로, 공격자는 손쉽게 악의적인 AP의 연결을 유도할 수 있다. 사용자가 이블 트윈에 연결되면 공격자는 중간자 공격

(Man In The Middle Attack)을 수행할 수 있다. 중간자 공격이란, 사용자가 알지 못하게 정상적인 공유기 사이의 통신을 중계하면서 통신을 엿듣거나 변조하는 것을 뜻한다. 공격자는 사용자의 각종 웹사이트의 비밀번호나 은행 계좌번호 등을 알아내기 위한 기법인 피싱(Phishing)을 시도할 수 있으며, 이 같은 특성으로 인해 적극적 공격(Active Attack)으로 분류된다. 이블 트윈은 인가되지 않은AP의 설치가 선행되어야 한다. 결국 공격자가 인가되지 않은AP 설치를 시도할 때 이를 탐지하고 대처한다면, 이블 트윈을 막을 수 있다. 하지만 여전히, 공격자가 인가된 AP와 동일한 맥 주소로 이블 트윈을 생성할 시에 문제가 발생한다. 대부분의 장치 및 소프트웨어에서 네트워크 내에서 동일한 맥 주소를 갖는 두 개의 AP가 존재한다고 인식하지 못하는 것과 출력 신호가 강한 이블 트윈만 인식하기 때문이다.

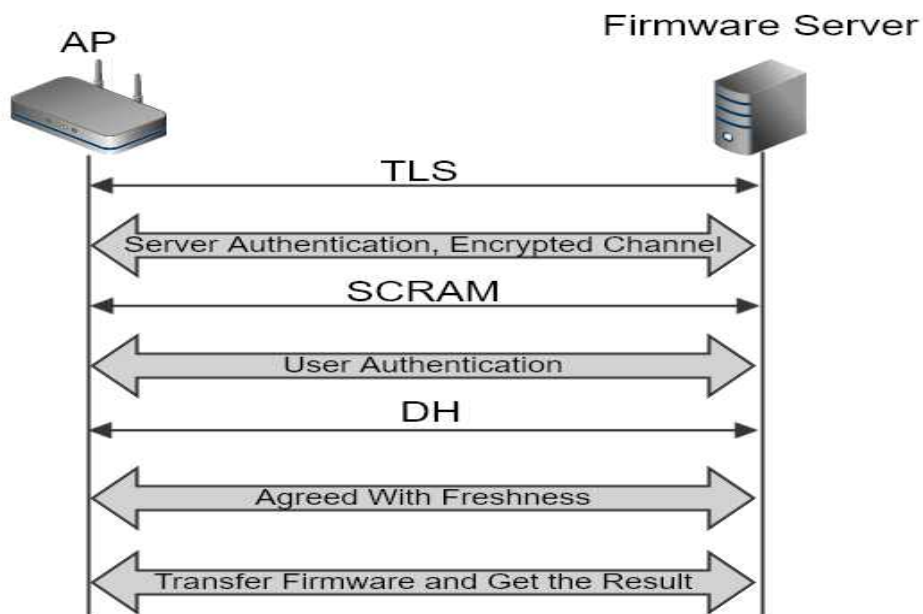


[그림 3-55] 로그 AP 공격원리

제 4 장 안전한 펌웨어 업데이트 방법

앞서 3장 4절에서 본 논문에서 발견한 펌웨어 업데이트 취약점이 얼마나 치명적인지 알아보았다. 경북대의 이임영 교수는 안전한 펌웨어 업데이트를 위하여 보안 프로토콜을 제시했다.[16] 제안된 프로토콜은 TLS(Transport Layer Security) 및 SCRAM(Salted Challenge Response Authentication Mechanism)을 사용하여 안전한 펌웨어 업데이트를 제공할 수 있는 인증 프로토콜을 제시했다. 또한, fresh함을 보장하기 위해 DH(Diffie-Hellman) 키 교환을 사용했다. TLS를 이용하여 펌웨어 서버 인증서를 확인함으로써 중간자 공격을 막을 수 있다. 또한 SCRAM을 이용하여 서버에 사용자 인증을 통해서 상호인증을 할 수 있게 해준다.

[그림 4-1]은 4 단계로 구성된 프로토콜을 보여준다. 가정으로는 미리 공유된 대칭 키가 존재하고 모든 서버와 WAP에 공개 키와 개인 키 쌍이 있다고 가정했으며, 대칭 키는 TLS 프로토콜의 일시적인 DH 키 협약 프로토콜에 의해 계산된다고 가정했다.



[그림 4-1] 경북대 이임영 교수가 제안한 펌웨어 업데이트 프로토콜

- ① 첫 번째 단계에서 WAP는 펌웨어 서버 인증서를 확인하고 TLS를 사용하여 암호화 된 채널을 설정한다. 이를 통해 MITM 공격으로부터 보호한다.
- ② 두 번째 단계에서는 WAP는 SCRAM을 사용하여 펌웨어 서버를 인증한다. 여기서 TLS는 인증서 기반이지만 SCRAM은 암호를 기반으로 인증한다.
- ③ 세 번째 단계에서는 익명의 DH 키 협약을 통해 키를 공유한다. DH는 fresh함을 결정해 준다.
- ④ 네 번째 단계에서 펌웨어 서버는 펌웨어에 fresh함을 추가하여 이를 WAP에 전송한다. The WAP은 펌웨어의 fresh함과 세 번째 단계에서 계산 된 fresh함을 비교하여 일치하면 업데이트를 진행한다.

제 5 장 결 론

본 논문에서는 국내에서 78%의 높은 점유율로 사용되어지고 있는 A사의 무선 공유기 펌웨어 업데이트 취약점에 대해 분석하였으며, 정적분석을 통해 A사의 무선 공유기 자체에서 업데이트 서버를 이용하지 않고 펌웨어 파일을 CRC32 Checksum값으로 자가 검증하여 이 값을 악의적으로 수정했을 경우 업데이트가 적용이 되어 공격자가 관리자 권한을 탈취할 수 있는 취약점을 발견하였다. 관리자 권한 탈취 후 발생 가능한 공격으로 A사 무선 공유기의 자체에 /tmp 디렉터리에서 파일을 4MB이상 수용 시 재부팅이 되는 취약점을 이용하여 반복된 파일 업로드를 이용한 DoS(Denial of Service) 공격구현을 하였으며, 로그AP의 공격 유형들인 인가되지 않은 AP, 이블 트윈, 소프트 AP에 대해 조사함으로써 취약점의 위험성에 대해 서술했다.

또한, 기존에 무선 공유기에 사용되고 있는 보안 기술과 소프트웨어 스마트 업데이트에 사용되는 기술들을 조사하여 펌웨어 업데이트 시 발생할 수 있는 취약점에 대한 대응 방안을 제안하고, 암호화된 통신 및 업데이트 인증 서버를 활용하여 CRC32 Checksum을 악의적으로 변경하는 취약점에 대응할 수 있을 것으로 예상된다.

추후 연구로는 무선 공유기 펌웨어 업데이트를 위해 제안된 보안 프로토콜에 대한 보안성 검증이 되어있지 않으므로 AVISPA[17], Proverif[18] 그리고 Scyther[19]와 같은 정형화 보안 분석 도구나 인증 프로토콜을 정형화된 방식으로 기술하고 분석하는 양상논리인 BAN 논리(Burrows-Abadi-Needham logic)[20]을 이용하여 제안된 프로토콜에 대한 보안 검증을 수행할 예정이다.

참 고 문 헌

- [1] 김태형, 보안뉴스, “공유기 해킹 비상! 무선 네트워크 보안이 중요한 이유”, <http://www.boannews.com/media/view.asp?idx=45285>, 2015. 02
- [2] 뉴시스, “와이파이서 ‘개인정보 줄줄’...불안한 스마트폰 사용자들”, <http://news.naver.com/main/read.nhn?mode=LSD&mid=sec&sid1=101&oid=003&aid=0003837491>, 2011. 05
- [3] 디지털타임스, “IoT 취약점 신고 건수 362건...3배 증가”, http://www.dt.co.kr/contents.html?article_no=2017032802109960041001, 2017. 03.
- [4] KISA, ‘알기쉬운 무선랜 보안 안내서’, 2010. 10.
- [5] KISA, ‘알기쉬운 공중 무선랜 보안 안내서’, 2011. 12.
- [6] 오민준, “2016년 1~4월 유무선 공유기 판매동향, 추천 공유기는?”, <http://news.danawa.com/view?boardSeq=63&listSeq=3169322&site=1>, 브레인박스, 5월, 2016년
- [7] <http://www.compuzone.co.kr/main/main.htm>
- [8] 오민준, “2014 컴퓨존&브레인박스 BEST BUY&CHOICE PRODUCT AWARD“, https://www.brainbox.co.kr/bbs/board.php?bo_table=review&wr_id=5120, 브레인박스, 1월, 2015년
- [9] <https://www.wireshark.org/download.html>
- [10] <https://www.hex-rays.com/products/ida/support/download.shtml>
- [11] http://iptime.com/iptime/etc/diff_prd.php, 6월, 2017년
- [12] Embedded XINU, “TRX”, http://xinu.mscs.mu.edu/TRX_header, 6월, 2011년
- [13] “Firmware-Mod-Kit 사용하여 펌웨어 분석하기”, <http://apollo89.com/wordpress/?p=6339>, 6월, 2014년
- [14] Wikipedia, “cramfs“, <https://en.wikipedia.org/wiki/Cramfs>, 8월, 2016년
- [15] 이경목, 이명건, 조성제, “무선 공유기 펌웨어 취약점을 통한 DoS 공격 시나리오,” 한국정보과학회 학술발표 논문집, pp. 75-77, 12월, 2015년
- [16] Jaejin Jang, Y.Jung, “Sustainable and Practical Firmware Upgrade for

Wireless Access Point Using Password-Based Authentication”,
Sustainability, 8권 9호, pp. 876, 6월 2016년

[17] <http://www.avispa-project.org/>

[18] <http://prosecco.gforge.inria.fr/personal/bblanche/proverif/>

[19] <https://www.cs.ox.ac.uk/people/cas.cremers/scyther/>

[20] Michael Burrows, Martin Abadi, Roger Needham, “A Logic of
Authentication”ACM Transactions on Computer Systems, 8권 1호 pp.
13-36, 2월 1990년

감사의 글

길고도 짧은 대학생활을 마무리 하며 함께 고생하고 관심과 격려를 아끼지 않았던 모든 분들에게 감사의 글을 남깁니다. 학부시절 훌륭하신 정보보호학과 교수님들의 가르침 속에 보안이라는 학문을 배우며 이렇게 값진 결실을 맺게 되었습니다. 무엇보다도 이 논문이 있기까지 부족한 저희를 열정적으로 지도해주고 바쁜 일정 속에서도 논문지도에 많은 시간을 할애해주신 유일선 교수님께 진심으로 감사드립니다. 또한 학부 발전을 위해 힘써주시고 논문심사를 진행해주신 염홍열 교수님, 임강빈 교수님, 이선영 교수님, 서정택 교수님께 깊은 감사드립니다.

또한 이 논문을 위해 아낌없는 조언과 논문첨삭에 힘써준 대학원생인 권순현, 김지운 학우에게 조금이나마 감사의 인사를 전하고 싶습니다.

마지막으로 지난 1년간 함께 고생한 팀원들이 바쁜 와중에 각자의 업무를 충실히 해주었기에 2017년 공과대학 학술제에서 은상이라는 값진 결과를 얻어 너무나도 기쁩니다. 모두들 각자 계획한 꿈이 이루어지기를 기원하고, 대학 4년간 배운 지식과 경험을 바탕으로 각자 원하는 분야에서 자랑스러운 순천향대학교 정보보호학과 졸업생으로 성장하길 바라면서 감사의 글을 마칩니다. 다시 한 번 이 논문을 위해 아낌없는 조언과 관심을 주신 모든 분들께 진심으로 감사드립니다.